Defence Technology xxx (xxxx) xxx

Contents lists available at ScienceDirect

Defence Technology

journal homepage: www.keaipublishing.com/en/journals/defence-technology

Temporal flow-based dynamic graph neural network with robustness interpretation for DoS/DDoS IoT intrusion detection

Teresa Jency Bala [a], Anindya Nag [b], Chander Prabha [c], Sujit Biswas [d], Anupam Kumar Bairagi [a,*], Mohammed Al-Maitah [e], Mohima Chowdhury [f], Walid El-Shafai [g,**]

[a] Computer Science and Engineering Discipline, Khulna University, Khulna, 9208, Bangladesh
[b] Department of Environmental Science, Informatics and Statistics, Ca' Foscari University of Venice, Mestre, 30170, Venice, Italy
[c] Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, 140401, India
[d] School of Science and Technology, University of London, City St George's, London, EC1V0HB, United Kingdom
[e] Computer Science and Engineering Department, College of Applied Studies, King Saud University, Riyadh, 11451, Saudi Arabia
[f] Department of Mathematics and Statistics, University of Helsinki, Helsinki, 00014, Finland
[g] Department of Electronics and Electrical Communications Engineering, Faculty of Electronic Engineering, Menoufia University, Menouf, 32952, Egypt

ARTICLE INFO

Article history:
Received 22 April 2026
Received in revised form
4 June 2026
Accepted 15 June 2026
Available online xxx

Keywords:
GNN
GATv2
Intrusion detection
DoS/DDoS classification
Adversarial training
IoT security

ABSTRACT

With the increasing use of Internet of Things (IoT) devices, volumetric network intrusions have expanded significantly. Various detection systems for network intrusions face complications in distinguishing between Distributed Denial-of-Service (DDoS) attacks and single-source Denial-of-Service (DoS) attacks an aspect that remains insufficiently investigated. Conventional Deep learning (DL) approaches rely on flat feature representations that often fail to capture underlying distributed topological structure inherent to multi-source flooding attacks. We propose adversarial-trained Graph Attention Network v2 (GATv2) framework that models network flows as temporal graphs, enabling topology-level discrimination of attack origins. A subset of network traffic from TON_IoT dataset of one million flow with DoS, DDoS, and benign classes, is represented as flow-level graph with directed edges, encoding node and edge features derived *via* entropy-aware feature engineering pipeline. We introduce source dispersion entropy, Gini concentration coefficients-based graph node features and the output yielding a Cohen's *d* separation score of 2.486 between DoS and DDoS classes. These temporal PyG graphs are examined with graph level feature selection using Kruskal-Wallis H-test, MI filtering, and Boruta wrapper selection to reduce the feature set. The GATv2 classifier was trained with test balanced accuracy of 0.9996, macro *F*1-score of 0.9979, MCC of 0.9956 and hardened *via* projected gradient descent (PGD) adversarial training with a progressive ε schedule, maintaining a balanced accuracy of 0.9698 at $\varepsilon = 0.20$ and 0.9558 at $\varepsilon = 0.35$. *GNN* (Graph Neural Networks) *Explainer* identifies most attribution-significant subgraph structures for each traffic class, providing interpretable insights into attack topology.

© 2026 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

The interconnected IoT ecosystem functions as a fundamental element of present-day digital infrastructure by enabling connections between various devices such as sensors, cameras, industrial controllers, smart appliances, and medical equipment [1]. The implementation of automated systems together with data-based decision-making methods leads to an increase in potential security vulnerabilities [2]. IoT network devices are functionally

* Corresponding author.
** Corresponding author. Computer Science and Engineering Department, College of Applied Studies, King Saud University, Riyadh, 11451, Saudi Arabia.
E-mail addresses: anupam@cse.ku.ac.bd (A.K. Bairagi), walid.elshafai@el-eng.menofia.edu.eg (W. El-Shafai).

Peer review under the responsibility of China Ordnance Society.

https://doi.org/10.1016/j.dt.2026.06.002

2214-9147/© 2026 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Please cite this article as: T.J. Bala, A. Nag, C. Prabha *et al.*, Temporal flow-based dynamic graph neural network with robustness interpretation for DoS/DDoS IoT intrusion detection, Defence Technology, https://doi.org/10.1016/j.dt.2026.06.002

constrained, with limited computational resources and rely on multiple communication protocols that lack proper standardization and built-in security mechanisms. These protocols prioritize operational efficiency over secure communications, thereby expanding the potential attack surface and enabling more frequent cyberattacks. Distributed Denial of Service (DDoS) and Denial of Service (DoS) attacks represent two of the most significant threats targeting IoT networks [3]. Fundamentally, both attack vectors aim to exhaust target resources such as network bandwidth, connection states, or application-layer processing capacity to render services unavailable to legitimate users. However, from an operational, structural, and defensive standpoint, they are entirely distinct phenomena [4]. A DoS attack originates from a single source host that overwhelms a target service, host, or network resource with malicious traffic [3]. In contrast, a DDoS attack achieves the same objective through coordinated traffic generated by multiple compromised devices distributed across a network, often organized as a botnet [4]. Consequently, DoS attacks exhibit a centralized attack origin, whereas DDoS attacks are characterized by distributed and coordinated traffic sources. As a result, identifying and mitigating the attack source is generally more straightforward in DoS scenarios than in DDoS attacks. The distinction carries forensic and legal significance, as a DoS event implicates a specific originating machine, while a DDoS event signals the existence of a coordinated threat, or a contracted attack-as-a-service operation [4] requiring different investigative responses. Despite this operational divergence, the majority of existing network intrusion detection benchmarks and DL classifiers conflate both attack types under a common volumetric or flooding label. This conflation effectively eliminates the discriminative signal required to drive appropriate defensive actions [5], leaving automated systems unable to distinguish between single-source attacks and coordinated distributed campaigns.

Existing intrusion detection approaches are commonly divided into signature-based and anomaly-based methods. Signature-based cannot detect novel attack variants while anomaly-based systems experience excessive false positive rates when operating under actual IoT network conditions [6]. Conventional machine learning (ML) techniques require manual feature selection and treat flows as independent samples, ignoring inter-flow relationships. Deep learning (DL) [7] models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) improve representation learning by capturing nonlinear and temporal patterns in traffic data, but still operate on flattened flow representations of traffic, remaining blind to the coordinated topological structure that distinguishes DDoS from DoS traffic [8]. Graph Neural Networks (GNNs) offer a natural framework for modeling network flows as nodes and encoding communication relationships as edges, enabling detection of coordinated multi-flow attack patterns [9,10]. Recent research proposes graph-based solutions that model flows, devices and their interactions as graphs, enabling message passing across relational neighborhoods (e.g., GNN, GraphSAGE, E-GraphSAGE) [11]. For instance, edge-enhanced GraphSAGE models can address issues of ineffective node feature initialization [12] and fail to use edge features properly. The adversarial robustness of GNN-based NIDS [11], is typically assessed using artificial disturbances which create a false impression of their protection abilities in actual situations. The systems require static graph construction methods and they do not have strong feature pipelines that can handle traffic while they fail to provide clear explanations [13] that support operational decisions. By bridging these gaps the main contributions of this work are summarized as follows:

- We frame DoS/DDoS discrimination as a graph, explicitly exploiting source-node distribution as the primary structural formulation absent from existing IoT intrusion detection benchmarks.
- We propose a temporal flow-as-node graph framework connecting flows *via* semantically typed directed edges in order to capture patterns and coordinated flows within temporal sliding windows.
- We introduce entropy-aware features for DoS/DDoS separation incorporating source dispersion entropy, Gini coefficient, HHI, and multi-scale temporal ratios.
- We developed a GATv2 graph classifier hardened *via* a progressive PGD adversarial training curriculum, retaining balanced accuracy at high attack strength while standard baseline degrades.
- We provided explainability *via* a unified GNNExplainer attribution jointly ranking node and edge features, exposing structurally interpretable flow-network.

This study proceeds with Section 2 reviewing prior work on IoT intrusion detection. Section 3 describes the methodology including temporal sampling, feature engineering pipeline, flow-to-flow graph construction, model architecture, adversarial training, and the explainability framework. Section 4 presents data analysis results, adversarial training and a comparative analysis. Section 5 concludes the study and outlines future directions.

2. Literature review

IoT intrusion detection research has several interconnected directions due to the heterogeneous and adversarial nature of network traffic. We have reviewed areas relevant to DoS/DDoS detection, GNN approaches, adversarial robustness, and explainability.

2.1. Feature engineering and conventional DL for IoT-NIDS

Early IoT intrusion detection relied on core network components like packet size, inter-arrival time, and protocol flags as the basis for baseline classifiers and public benchmarks. Using sliding window statistics together with entropy-based descriptors to analyze temporal dynamics which combined three machine learning techniques Random Forest (RF) and Isolation Forest (IF) and LSTM autoencoders to achieve $F1$ scores above 0.96 [14]. The transition to graph-based models feature space to include structural attributes node degree, betweenness centrality, and edge directionality with surveys identifying heterogeneous graph constructions that combined temporal convolutions and attention-weighted edges [15]. E-GRACL [16] enriches edge features with global attention and modulation mechanisms to capture subtle traffic correlations.

2.2. GNN for IoT networks

GNNs have become the dominant pattern for modeling relational nature of IoT traffic. Graph autoencoder approaches demonstrated suitable accuracy efficiency trade-offs on large-scale IoT datasets [11]. Trust aware extensions fuses multi-dimensional trust scores into edge weights, allowing adaptive message propagation and yielding more than 0.94 accuracy on EDGE IIoTSET and IoT 23 while outperforming earlier GNN baselines [17]. Graph Attention Network (GAT) based architectures confirms the node-level attention captures subtle attack patterns

in heterogeneous IoT environments [13]. BS-GAT [18] addressed uneven neighbour aggregation *via* weighted behavioural similarity edges, and dynamic Graph Convolutional Network (GCN) [19] achieved *F*1 of 0.95 on DoS/MiTM through real-time graph evolution.

2.3. Adversarial robustness and explainability

The security improvement method for GNN-based detectors uses adversarial training, which operates as a min-max optimization process that adds worst-case perturbations to training batches [14]. Zhong et al. [20] independently conclude in their surveys that no GNN-NIDS study simultaneously addresses flow-as-node graph construction, temporal edge semantics, and adversarial robustness for fine-grained DoS/DDoS classification. Later, Yan et al. [21] combined hierarchical edge-GAT with PGD training across four benchmarks, achieving 0.8438 adversarial accuracy. On explainability, GNNExplainer [22] provides post-hoc subgraph attribution that is increasingly applied in security contexts where transparency is required for regulatory compliance [23]. The X-GANet framework [24] extended this with a Cross-Diffused Attention Transformer and entropy-driven masking, while SHAP-based explanations [25] demonstrated protocol-awareness in non-graph classifiers. This study proposes a robust and interpretable intrusion detection framework for DoS and DDoS threats that integrates GATv2 [26] model with adversarial robustness, and an understanding of graphs. Table 1 presents an overview of research studies that examine the implementation of GNN technology across various domains of IoT network research.

3. Methodology

The proposed research aims to develop a robust and explainable GNN-based framework for IoT networks. The overall methodology follows a systematic multi-stage pipeline from dataset acquisition to model explainability, combining both data-driven experimentation and graph-based modeling. The methodology is presented in Fig. 1.

3.1. Dataset and preprocessing

The experiments in this study use the publicly available TON_IoT dataset introduced by Moustafa [28], designed to support intrusion detection research across heterogeneous cyber-physical environments by integrating IoT telemetry, network traffic, and more into a unified security benchmark. The network traffic component was originally collected *via* controlled attack simulations, raw packet captures processed by the Zeek framework [29] to produce structured flow records, which the dataset author released as pre-processed CSV files. In our study, we used those pre-processed records directly. The full dataset contains approximately 8.54 million flow records spanning multiple attack categories.

This flow-level dataset is particularly suited to graph-based modeling, as it carries host-centric network flows as isolated, independent variables. To focus on volumetric attacks, which represent the primary threat to network availability in IoT deployments, we retained three traffic classes: DDoS (6,081,852 flows), DoS (1,815,151 flows), and normal (638,061 flows), excluding man-in the-middle, ransomware, scanning, backdoor, and password-based attacks, as they follow different traffic generation mechanisms. After filtering, the necessary columns for network propagation were chosen, and exact de-duplicate rows records identical across all feature columns to prevent memorization bias, retaining only the first occurrence of each unique record. Each record contains 15 raw features, including timestamps, IP addresses, port numbers, protocol, connection state, flow duration, byte and packet counts, and attack label.

3.2. Feature engineering

Naive random sampling is unsuitable because it cannot capture the temporal structure of attack campaigns, which is critical for modeling DoS and DDoS behaviors. We down-sampled to 1,000,000 flows using a hybrid temporal sampling strategy designed to preserve diversity across the full capture window while maintaining controlled class proportions of 0.35 DoS, 0.35 DDoS, and 0.30 Normal traffic. The sampling procedure applies to each class independently. For DoS flows, 0.60 of the class budget (349,995 flows) is allocated *via* fixed-width temporal binning (30 bins), and the remaining 0.40 is drawn *via* quantile-density sampling (15 bins). DDoS flows follow the same hybrid approach with 40 fixed-width bins and 20 quantile bins. Normal flows are sampled *via* stratified quantile binning (20 bins). Fig. 2 illustrates

Table 1
Summary of the recent works on the network of IoT.

Ref.	Dataset used	Objective	Method	Results	Limitations
[27]	TON_IoT, simulated IoT data	Develop ML framework	Hybrid RF + IF + LSTM	$F1 > 0.96$	No graph structure
[9]	NF-ToN IoT, NF-BoT IoT	GNN with attribute similarity	GraphSAGE, Top-K similarity graph (TKSGF)	Binary *F*1: 0.9852, multi-class *F*1: 0.8404	No edge features, no explainability
[10]	Small benchmark NSL-KDD dataset	GAT to create a detection system	GAT + Euclidean similarity graph	Binary Acc: 0.98, multi-class Acc. 0.992	Small outdated dataset, no adversarial evaluation
[18]	NF-BoT-IoT-v2, NF-ToN-IoT-v2	Improved GNN with multiclass	BS-GAT with weighted behavioural edges	Binary $F1 > 0.99$, multi-class Acc >0.93	no adversarial hardening
[19]	Simulated (DoS, Spoofing, MiTM), TON_IoT	GNN-based IDS	Dynamic GCN, real-time graph evolution	$F1 = 0.95$, AUC-ROC = 0.98	No explainability
[24]	CIC-IDS2017, UNSW-NB15	X-GANet with explainability	X-GANet with XDA Transformer + entropy masking	Acc: 0.998/0.989	Relies on graph construction quality
[25]	Edge-IIo tset	Explainable detection for zero-day attacks	Multi-head attention + BiLSTM + SHAP	Binary Acc: 0.9999; multi-class Acc: 0.9834	Non-graph, no adversarial robustness
[16]	IoT network traffic datasets	Improve relational feature learning	Edge-enhanced graph attention with global attention modulation	Better capture of subtle traffic correlations and edge relationships	No explainability, or robustness
[17]	EDGE-IIoTset, IoT-23	Trust-aware intrusion detection	GNN with trust-weighted message propagation	Accuracy >0.94	No adversarial defense, limited interpretability
[21]	Four network intrusion benchmarks	Improve robustness of GNN-based IDS	Hierarchical edge-GAT + PGD adversarial training	Adversarial accuracy = 0.8438	No explainability, not specialized for DoS/DDoS

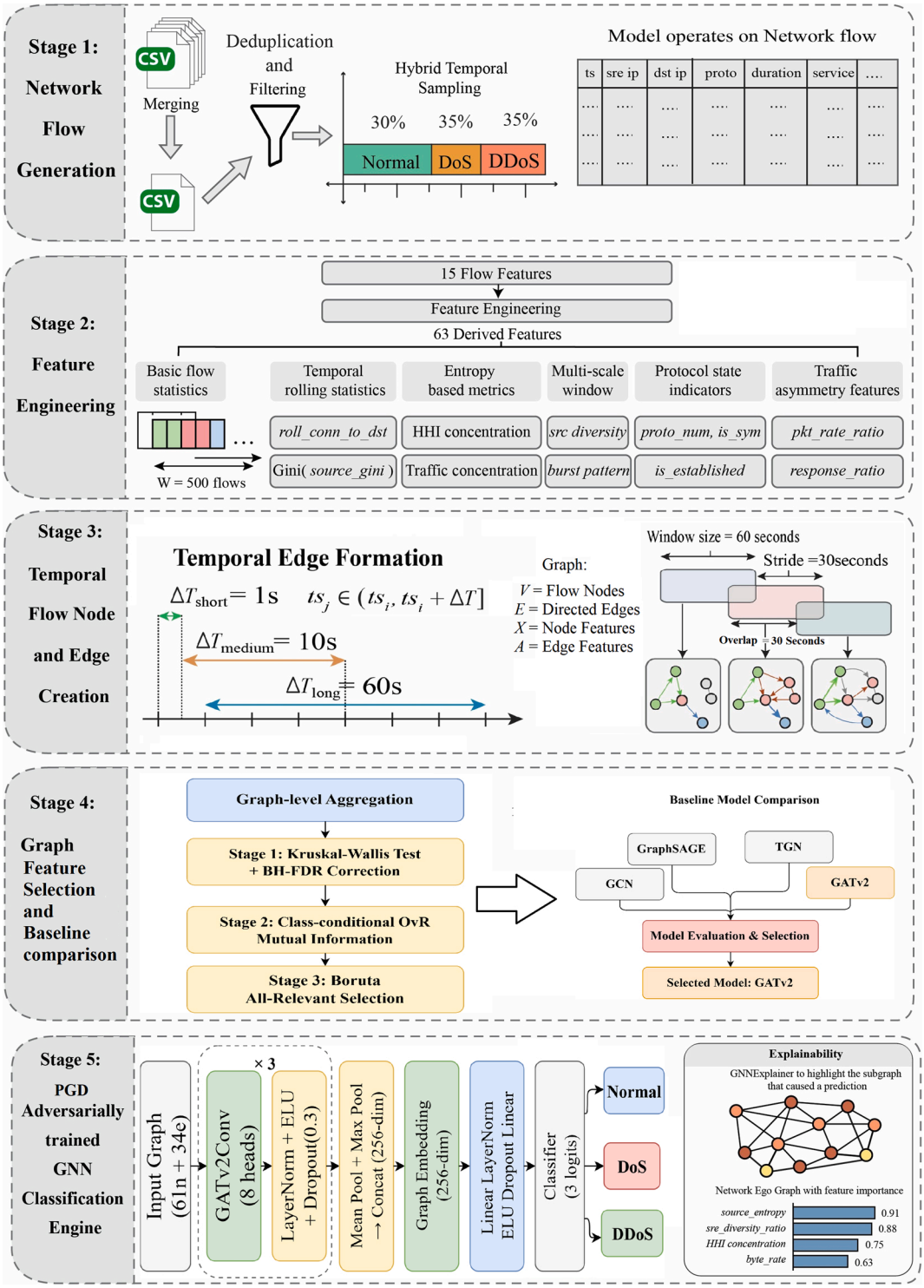


Fig. 1. The Five-stage research workflow with preprocessing, feature engineering, graph constructions, feature selectionand PGD-hardened GATv2 model classification.

the temporal flow distribution of the traffic.

A balanced working dataset of 9,99,993 flows was created that spans a total capture window of 27.21 d (2 April 2019 09:45:58 UTC to 29 April 2019 14:45:56 UTC, ~653 h). Normal traffic spans the full window at 459.4 flows/hour (mean IAT 7.84 s). In comparison, DoS is concentrated in an 8.61-h burst at 40,645.9 flows/hour (mean IAT 0.089 s), and DDoS occupies a 16.74-h window at 20,900.7 flows/hour (mean IAT 0.172 s); both attack classes are two to three orders of magnitude more intact than benign traffic. After sampling, 15 raw columns are transformed into 67 derived features spanning into semantic categories: basic flow metrics, rate features, asymmetry and balance indicators, packet-level statistics, protocol and state flags, connection characteristics, port features, and rolling-window temporal aggregates over 500-flow segments capturing victim-side statistics (connections received, unique source IPs, SYN and RST rates) and attacker-side statistics (unique

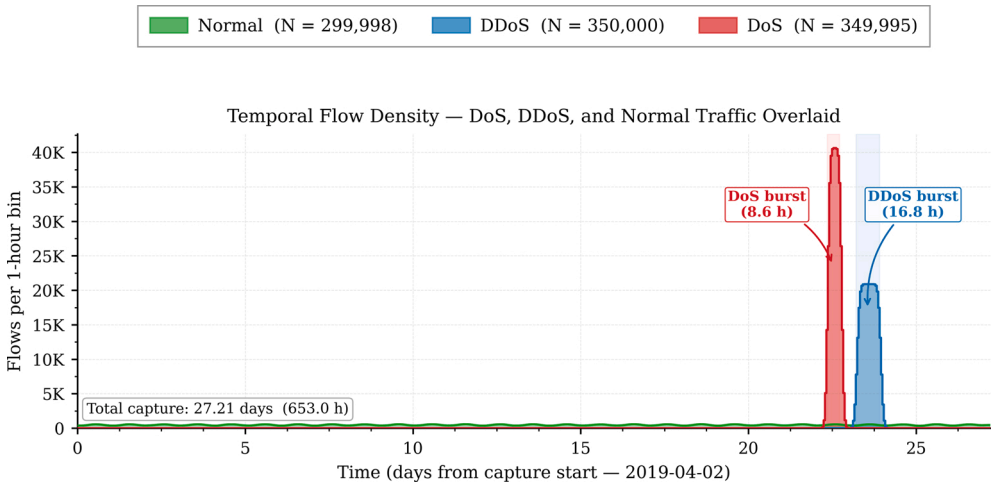


Fig. 2. Temporal flow density of Normal, DoS, and DDoS traffic across 27.21 days capture window.

destinations contacted, outgoing packet rate). The fundamental differentiator between DoS and DDoS is the cardinality and distribution of the source IP address space. We apply Shannon entropy to the source IP distribution at each destination. Let $S = \{s_1, s_2, \ldots, s_k\}$ be the set of unique source IP addresses in a window, and $p(s_i)$ be the probability of occurrence of the ith source IP.

$$H(S) = -\sum_{i=1}^{k} p(s_i)\log_2 p(s_i) \tag{1}$$

Source dispersion entropy applies Shannon entropy to the source IP distribution at each destination, yielding $H(S) \approx 0$ for single-source DoS and $H(S) = \log_2(S)$ for distributed DDoS traffic. A scale-invariant, evasion-resistant discriminator, since an attacker cannot reduce DDoS entropy without abandoning the distributed architecture that defines the attack (Eq. (1)). In the study by Liu et al. [30] pioneered a Gini-based DDoS detection, where the coefficient of source IP distributions effectively separates attacks from normal traffic. Gini = 0 indicates uniform distribution (DDoS) and Gini = 1 indicates full concentration (DoS). For sorted source counts, x_i (where $x_1, x_2, \ldots, x_n$), is computed in (Eq. (2)).

$$G = \frac{2\sum_{i=1}^{n} i.x_i}{n\sum_{i=1}^{n} x_i} - \frac{n+1}{n} \tag{2}$$

Applied to network traffic, the Herfindahl-Hirschman Index (*HHI*) measures whether packets are monopolistically concentrated in one source (DoS) or competitively distributed across many sources (DDoS) [31]. $HHI_{DoS} > HHI_{DDoS}$, for a single source $HHI = 1$, while for k equally contributing sources $HHI = 1/k$. HHI is particularly sensitive to centralized flooding behavior. Multi-scale temporal features extend these measures across three nested window sizes (100, 500, 1000 flows), computing source diversity and connection rates at each scale and their cross-scale ratios, capturing the observation that DoS attacks exhibit consistently low diversity ratios while DDoS patterns fluctuate across timescales according to botnet coordination mechanisms [32]. This algorithm 1 extracts multiple temporal features at different time periods. The model will be able to capture both transient bursts and sustained trends, a distinction vital for attack classification [33].

Algorithm 1 Multi-scale temporal feature extraction.

Input: D: Network flow dataset with columns src_{ip}, dst_{ip}
W: Set of window sizes {100, 500, 1000}
Output: F: Multi-scale feature matrix
1: Encode src_{ip} and $dst_{ip} \rightarrow src_{enc}$, dst_{enc} using LabelEncoder.
2: temp ← {src_{enc}, dst_{enc}}.
3: Initialize empty map Features, F.
4: For each window size $w \in W$ do
5: For each destination d in dst_{enc} do
6: Let S_d = sequence of src_{enc} where $dst_{enc} = d$
7: For each time index t in S_d do
8: Define rolling window:
9: $W_w(t)$ = last w flows ending at t
10: Compute unique source count:
11: $U_w(t)$ = | unique sources in $W_w(t)$ |
12: Compute connection rate:
13: $R_w(t) = \frac{|Ww(t)|}{w}$
14: End For
15: End For
16: Store U_w and R_w in F
17: End For
18: Compute cross-scale ratios:
19: $\text{Ratio}_{100_500}(t) = \frac{U_{100}(t)}{U_{500}(t)+1}$
20: $\text{Ratio}_{500_1000}(t) = \frac{U_{500}(t)}{U_{1000}(t)+1}$
21: Return feature matrix Fs

3.3. Feature analytics and statistical validation

A class-wise feature distribution is used to calculate separation scores through standardized effect size measurements. One-way ANOVA F-statistic test presented significant differences among classes. In analysis using Mutual Information (MI) and RF, the source Gini coefficient achieves the highest Cohen's d separation score of 2.486 between DoS and DDoS classes, followed by src_diversity_ratio_500_1000 (d = 1.200) and traffic_concentration_hhi (d = 0.964). In Table 2, average separation scores above 0.5 indicate strong discriminative power, maximum separation identifies individual features within each category, and average F-statistic confirms the significance of between-class differences, with all categories yielding $P < 0.001$.

Table 2
Feature categories and their analytics.

Feature category	#Features	Avg. separation	Max separation	Avg. *F*-statistic
Port features	2	2.117	2.64	654,503.3
Temporal-attacker perspective	3	1.763	2.315	440,691.2
Temporal-victim perspective	6	1.272	2.68	450,238
Protocol & state	9	1.063	2.848	459,452
Temporal-flow perspective	4	0.962	1.88	226,346.5
Connection characteristics	5	0.906	1.269	285,340.1
Asymmetry & balance	5	0.879	1.413	176,901.4
Source distribution	3	0.757	1.447	138,135.2
Attack signatures	5	0.673	1.462	134,474.4
Burst patterns	1	0.416	0.416	32,143.58
Multi-scale temporal	8	0.406	0.732	49,614.43
Rate features	6	0.191	0.364	8923.676
Packet size indicators	2	0.173	0.253	8285.842
Packet-level features	3	0.173	0.199	2543.243
Basic flow metrics	5	0.096	0.358	1138.645

3.4. Flow-to-flow graph construction with temporal edge typing

Heterogeneous directed temporal graph $G = (V, E)$ representing each flow record as node $v \in V$ with behavioral/statistical/protocol features, and edges $e \in E$ capture structural/temporal relationships with edge types, weight (similarity), temporal gap flows (inspired from E-GraphSAGE [34] and FlowTransformer [35]) enabling GNN detection of multi-flow attack coordination missed by single-flow methods. The four edge types and their similarity thresholds were selected to encode the structural signatures that empirically differentiate the three traffic classes. Chained edges (threshold 0.65) capture sequential hop patterns and lateral movement in multi-step paths [36]. IP-reversal edges (threshold 0.70) encode bidirectional TCP handshake pairs characteristic of normal and low-rate DoS traffic [18]. Shared-destination edges (threshold 0.75) encode the fan-in convergence signature of DDoS, where many sources flood a single victim [37]. Shared-source edges (threshold 0.80) encode the fan-out signature of single-source DoS, where one attacker contacts many destinations. The three temporal scales (1, 10, 60 s) capture burst-level, inter-bot coordination, and campaign-level structure respectively, consistent with botnet detection literature recommending sub-second to minute granularity for volumetric flood detection [38]. Edges connect flow pairs (v_i, v_j) satisfying $ts_j \in (ts_i, ts_i + \Delta t]$ across three temporal scales: $\Delta t = 1.0/10.0/60.0$ s. Edges are chained connections ($dst_ip_{(i)} = src_ip_{(j)}$), shared source edges ($src_ip_{(i)} = src_ip_{(j)}$), shared destination edges ($dst_ip_{(i)} = dst_ip_{(j)}$), and IP-reversal edges ($src_ip_{(i)} = dst_ip_{(j)} \wedge dst_ip_{(i)} = src_ip_{(j)}$). Edge candidates require normalised cosine similarity > adaptive behavioural threshold (0.65 chained, 0.70 IP-reversal, 0.75 shared-dest, and 0.80 for shared-src). The resulting graph comprises 999,993 nodes and 17,683,986 edges. Each edge $e_{(ij)}$ carries a 37-dimensional feature with edge type and temporal window (encoded), time gap between flows, their behavioural similarity score, the raw feature values of both endpoint nodes, and absolute difference features ($|feat_i - feat_j|$) and composite weight normalizing similarity, proximity, and an attack-interaction. Edge timestamps are assigned as $ts_e = \max(ts_i, ts_j)$, derived by vectorized merging. Node features $X \in \mathbb{R}$ (999,993 × 68), excluding temporal/IP/labels, are numerically robust scaled. And 2 categorical (protocol/TCP) integer label encoded.

3.5. Per-class stratified temporal split

A per-class stratified temporal split for each of the three traffic classes independently, flows are sorted by timestamp, and the first 70/15/15% into train, validation, and test. The three classes occupy non-overlapping temporal ranges: normal (~565 h), DoS (~6.1 h), and DDoS (~95 h). A naive global split would produce validation sets with zero DoS graphs. A membership-based edge assignment, where each edge is assigned to the partition containing both its source and destination nodes. Only 151,467 cross-partition edges (< 0.01) are discarded.

Table 3 shows resulting partition statistics. Graph snapshots are generated *via* temporal sliding window of duration $W = 60$ s and stride $W/2$ (50% overlap). For each window $[t, t + W)$, nodes with timestamps within the window are retained as active and edges are included only if both endpoints are active and the edge timestamp satisfies $t_e \leq t + W$, enforcing strict causal ordering. Windows with fewer than 5 nodes or 3 edges are discarded. Labels follow a most-severe-wins with DDoS takes precedence over DoS, which takes precedence over Normal. This yields 15,817 labeled graph snapshots. The complete graph dataset is stored as PyTorch Geometric Data objects.

To verify partition integrity and address potential data leakage concerns, graph identity tuples defined by window start time, window end time, and label were confirmed mutually exclusive across all three partition pairs, with zero overlap in all pairwise comparisons ($\text{Train} \cap \text{Val} = 0$, $\text{Train} \cap \text{Test} = 0$, $\text{Val} \cap \text{Test} = 0$). Label distributions were consistent with recorded metadata across all splits, and feature dimensions were uniform at 68 node features and 37 edge features throughout. Temporal ordering was verified at the individual flow level for each class independently: the maximum training flow timestamp does not exceed the minimum validation flow timestamp, and the maximum validation flow timestamp does not exceed the minimum test flow timestamp, confirmed for Normal (train t_s_max = 1,556,262,277, validation t_s_min = 1,556,262,277), DoS (train t_s_max = 1,556,151,659, validation t_s_min = 1,556,151,660), and DDoS (train t_s_max = 1,556,244,432, validation t_s_min = 1,556,244,433). Apparent window-level boundary gaps of up to 45 s observed at partition boundaries are a structural artifact of the 60-s window duration with 30-s stride, where the final window of one partition nominally overlaps the first window of the next in calendar time while containing strictly disjoint flows by individual timestamp assignment, and do not constitute flow-level contamination. The RobustScaler applied to node features is fitted on the full node population prior to the train-validation-test split and is label-agnostic by definition, as it operates solely on median and interquartile range statistics without accessing class labels. Eighteen features showed IQR drift exceeding 0.15 between training and test partitions, the primary cause of which is class-temporal confounding: normal traffic spans the full 565-h capture window while attack classes are temporally concentrated burst phases, producing a different class composition in the test partition (0.86

Table 3
Graph distribution information.

Parameter		Training set	Validation set	Testing set
Total graphs		9370	2800	3647
Mean nodes per graph		149.3	106.5	81.4
Max nodes per graph		1991	1781	1338
Mean edges per graph		2319.9	1662.0	1285.7
Window duration (T_w)/s		60	60	60
Temporal overlap		0.50	0.50	0.50
Total nodes		699,993	149,998	150,002
Total edges		12,305,681	2,626,844	2,599,994
Class distribution	Normal	7274 (0.776)	2345 (0.838)	3145 (0.862)
	DDoS	1359 (0.145)	305 (0.109)	352 (0.097)
	DoS	737 (0.079)	150 (0.054)	150 (0.041)

Normal graphs). This drift does not introduce label information into the scaling parameters, and since all model variants in this study use the same scaler, relative performance comparisons remain valid throughout.

3.6. Graph-level feature selection

We apply a three-stage statistical feature selection pipeline to the training partition with graph-level aggregations (mean/max/std on 68 node and mean/max on 37 edge features). First, Kruskal-Wallis H-test with Benjamini-Hochberg FDR correction ($\alpha = 0.05$) identifies features with significantly divergent class distributions. Mutual information (global + one-vs-rest) then retains class-discriminative signals despite DoS imbalance (0.079). Finally, Boruta Random Forest (balanced 9K samples, 200 trees) selects all-relevant features *via* CONFIRMED ∪ (TENTATIVE ∩ KW ∩ MI > 0), yielding 61 node and 34 edge features after eliminating redundant rate aggregates, binary flags, and correlated multi-scale differences.

An audit formally verified the absence of direct label leakage in the feature set. The following columns were explicitly excluded from all node and edge features prior to model input: raw timestamps (*ts*), source and destination IP addresses (src_ip, dst_ip), port identifiers (src_port, dst_port), the graph node key (node_id), the target variable in its original string form (label, type), a target-derived binary indicator (is_attack), preprocessing artifacts (qbin, service), and the encoded target variable (label_encoded). Verification confirmed that zero direct leakage columns and zero identifier columns appear in either the 61 retained node features or the 34 retained edge features. Regarding the reviewer concern on semantic leakage, source-distribution features (source_entropy, source_gini, traffic_concentration_hhi, src_diversity_ratio, and related multi-scale variants) are present in the model. These are not direct label leakage: they are computed from raw packet counts and IP diversity statistics prior to label assignment, using the same computation a deployed intrusion detection system would apply, and are analogous in function to packet rate features in any volumetric flood detector. They are derived behavioral features, not target variables. Their discriminative contribution is bounded and quantified by removing all 14 source-distribution features reduces balanced accuracy by only 1.03 percentage points, confirming that the model does not simply re-implement source-distribution rules.

3.7. GNN model development

Each graph snapshot is classified using GATv2 [39], which extends GAT by applying the linear transformation W after concatenating source and target node representations, yielding dynamic attention coefficients.

Table 4
Symbol notation and description used in the TFA-GATv2 model.

Symbol	Dimension	Meaning
$G = (V, E, X, A)$	–	Input temporal graph snapshot per traffic window
$\boldsymbol{x}_i$	$\mathbb{R}^{61}$	Feature vector of flow node i
$\boldsymbol{e}_{ij}$	$\mathbb{R}^{34}$	Attribute vector of directed edge $i \to j$
$N(i)$	–	Set of in-neighbors of node i
$\boldsymbol{B}$	$\mathbb{Z}^{\lvert V\rvert}$	Batch assignment vector; $\boldsymbol{B}[v]$ = graph index of node v
C	3	Output classes: {DDoS = 0, DoS = 1, Normal = 2}
$d_x/d_e/d_h$	61/34/128	Node feature/edge attribute/hidden dimension
K	8	Number of attention heads (all layers)
D_{head}	16	Per-head dimension: $d_h/K = 128/8$
$\boldsymbol{h}_i^{(0)}$	$\mathbb{R}^{61}$	Initial node embedding: $\boldsymbol{h}_i^{(0)} = \boldsymbol{x}_i$
$\boldsymbol{h}_i^{(\ell)}$	$\mathbb{R}^{128}$	Node i embedding after GATv2 layer ℓ ($\ell \in \{1, 2, 3\}$)
$\boldsymbol{X}_{res}$	$\mathbb{R}^{\lvert V\rvert \times 128}$	Residual carry: Layer 1 output passed into Layer 2
$\boldsymbol{W}^{(\ell)}$	L1: $\mathbb{R}^{128\times 156}$/L2–3: $\mathbb{R}^{128\times 290}$	Shared linear weight in GATv2 layer ℓ
$\boldsymbol{a}^{(\ell)}$	$\mathbb{R}^{128}$	Learnable attention vector in layer ℓ
$\alpha_{ij}^{(\ell)}$	[0, 1]	Normalised attention weight; $\Sigma_{i \in N(j)}\, \alpha_{ij}^{(\ell)} = 1$
$LN^{(\ell)}$	–	LayerNorm after GATv2 layer ℓ (per node, feature dim)
$\boldsymbol{z}_{mean}$	$\mathbb{R}^{128}$	Global mean-pool embedding
$\boldsymbol{z}_{max}$	$\mathbb{R}^{128}$	Global max-pool embedding
$\boldsymbol{z}_G$	$\mathbb{R}^{256}$	Concatenated graph embedding: $[\boldsymbol{z}_{mean} \Vert \boldsymbol{z}_{max}]$
$\boldsymbol{W}_A, \boldsymbol{b}_A/\boldsymbol{W}_B, \boldsymbol{b}_B/\boldsymbol{W}_C, \boldsymbol{b}_C$	256 → 128/128 → 64/64 → 3	MLP layer weight matrices and bias vectors
p	0.30	Dropout probability (Layers 1, 2, MLP-A, MLP-B)
$\hat{\boldsymbol{y}}_G$	$\mathbb{R}^3$	Raw class logit vector (before softmax)
$\boldsymbol{p}_G$	$\mathbb{R}^3$	Class probability vector: softmax($\hat{\boldsymbol{y}}_G$)
y_{pred}	{0, 1, 2}	Predicted class label: $\text{argmax}_c\, \boldsymbol{p}_G[c]$

$$e_{ij} = \boldsymbol{a}^{\mathrm{T}}\,\mathrm{LeakyReLU}\Big(\boldsymbol{W}\left[\boldsymbol{h_i} \,\|h_j\| e_{ij}\right]\Big) \tag{3}$$

where a is a learnable attention vector, following (Eq. (3)), W is a shared weight matrix, h_i and h_j are source and target node embedding, and e_{ij} is the edge attribute vector. Unlike GAT, whose attention is provably static [26], GATv2 conditions attention jointly on both endpoints, critical for attack detection where an attacker-victim receive high attention while a benign edge from the same victim receives low attention. With baseline comparison on the GCN [40] lacks edge feature support and assumes undirected graphs, while GraphSAGE [41] applies uniform neighbour aggregation and cannot prioritize attacker-victim edges. Although TGN [42] handles time-based dynamics, it lacks the edge-specific attention required to prioritize attacker-victim connections. GATv2 resolves both limitations through dynamic attention weighting and is therefore adopted in this study.

3.7.1. Model architecture

The proposed Temporal Flow-Adversarial-Gatv2 (TFA-GATv2) comprises three stacked GATv2 message-passing layers, a dual-pool readout, and a three-layer MLP classifier. After three layers, each node embedding encodes information from its 3-hop neighborhood. Global mean pooling and max pooling are concatenated to form a 256-dimensional graph-level vector, passed through the MLP with ELU activations and dropout, to produce class logits.

Focal loss [43] replaces standard cross-entropy to address class imbalance, down-weighting examples, and focusing learning on harder DoS/DDoS boundary cases. Table 4 defines all symbols, and Algorithm 2 presents the complete forward pass.

3.7.2. Training configuration

Training uses the Adam optimizer [13] with initial learning rate 1×10^{-3}, weight decay 1×10^{-4}, and gradient clipping at norm 1.0. The learning rate is reduced on a plateau with a factor of 0.5 and patience of *5* epochs, with a minimum learning rate of 1×10^{-6}. Early stopping is applied with patience of 15 epochs on validation balanced accuracy with a batch size of *32*. Focal Loss is applied with $\gamma = 2.5$ and per-class weights of $\alpha \approx [1.0, 1.0, 1.17]$ for [DDoS, DoS, Normal] derived *via* square-root inverse class frequency to mitigate the severe class imbalance in which normal traffic constitutes 77.6% of training samples.

Algorithm 2 TFA-GATv2 architecture: forward pass.

Input: Graph batch: $G = (V, E, X, A, B)$; $X \in \mathbb{R}^{|V|\times 61}$, $A \in \mathbb{R}^{|E|\times 34}$
Output: $\hat{\boldsymbol{y}}_G \in \mathbb{R}^3$ (logits), $\boldsymbol{p}_G \in \mathbb{R}^3$ (probabilities), $y_{\mathrm{pred}} \in \{0,1,2\}$
Params: $d_h = 128$, $K = 8$, $d_{\mathrm{head}} = 16$, $P = 0.30$, $C = 3$
$\boldsymbol{h}_i^{(0)} \leftarrow x_i \;\forall\, i \in V$//INITIALIZATION
$\boldsymbol{x}_{\mathbf{res}} \leftarrow$ null
for each edge $(i \to j) \in E$://GATV2 L1 [61 → 128]
$\mathrm{concat}_{ij} \leftarrow [\boldsymbol{h}_i^{(0)} \| \boldsymbol{h}_j^{(0)} \| \boldsymbol{e}_{ij}] \in \mathbb{R}^{156}$//61 + 61+ 34 = 156
$\boldsymbol{e}_{ij}^{(1)} \leftarrow a^{(1)\mathrm{T}}.\mathrm{LeakyReLU}(\boldsymbol{W}^{(1)}.\mathrm{concat}_{ij})$
$\alpha_{ij}^{(1)} \leftarrow \mathrm{softma}\boldsymbol{x_i} \in N(j)(\boldsymbol{e}_{ij}^{(1)})$
$\boldsymbol{m}_i^{(1)} \leftarrow \Sigma_j \in N(i)\ \alpha_{ij}^{(1)}.\boldsymbol{W}\mathrm{v}^{(1)}.\boldsymbol{h}_j^{(0)}$
$\boldsymbol{h}_i^{(1)} \leftarrow \mathrm{ELU}(LN^{(1)}(\boldsymbol{m}_i^{(1)}))$
$\boldsymbol{h}_i^{(1)} \leftarrow \mathrm{Dropout}(\boldsymbol{h}_i^{(1)}, P)$
$\boldsymbol{x}_{\mathrm{res}} \leftarrow \boldsymbol{h}^{(1)}$
for each edge $(i \to j) \in E$://GATV2 L2 [128 → 128, RESIDUAL]
$\mathrm{concat}_{ij} \leftarrow [\boldsymbol{h}_i^{(1)} \| \boldsymbol{h}_j^{(1)} \| \boldsymbol{e}_{ij}] \in \mathbb{R}^{290}$//128 + 128 + 34 = 290
$\boldsymbol{e}_{ij}^{(2)} \leftarrow \boldsymbol{a}^{(2)\mathrm{T}}.\mathrm{LeakyReLU}(\boldsymbol{W}^{(2)}.\mathrm{concat}_{ij})$
$\alpha_{ij}^{(2)} \leftarrow \mathrm{softmax}_i \in N(j)\ (\boldsymbol{e}_{ij}^{(2)})$
$\boldsymbol{m}_i^{(2)} \leftarrow \Sigma_j \in N(i)\ \alpha_{ij}^{(2)}.\boldsymbol{W}\mathrm{v}^{(2)}.\boldsymbol{h}_j^{(1)}$
$\boldsymbol{h}_i^{(2)} \leftarrow \mathrm{ELU}(LN^{(2)}(\boldsymbol{m}_i^{(2)}) + \boldsymbol{x}_{\mathrm{resi}})$
$\boldsymbol{h}_i^{(2)} \leftarrow \mathrm{Dropout}(\boldsymbol{h}_i^{(2)}, P)$
for each edge $(i \to j) \in E$//GATV2 L3 [128 → 128, NO RESIDUAL]
$\mathrm{concat}_{ij} \leftarrow [\boldsymbol{h}_i^{(2)} \| \boldsymbol{h}_j^{(2)} \| \boldsymbol{e}_{ij}] \in \mathbb{R}^{290}$//128 + 128 + 34 = 290
$\boldsymbol{e}_{ij}^{(3)} \leftarrow \boldsymbol{a}^{(3)\mathrm{T}}.\mathrm{LeakyReLU}(\boldsymbol{W}^{(3)}.\mathrm{concat}_{ij})$
$\alpha_{ij}^{(3)} \leftarrow \mathrm{softmax}_i \in N(j)\ \boldsymbol{e}_{ij}^{(3)}$
$\boldsymbol{m}_i^{(3)} \leftarrow \Sigma_j \in N(i)\ \alpha_{ij}^{(3)}.\boldsymbol{W}\mathrm{v}^{(3)}.\boldsymbol{h}_j^{(2)}$
$\boldsymbol{h}_i^{(3)} \leftarrow \mathrm{ELU}(LN^{(3)}(\boldsymbol{m}_i^{(3)}))$
$\boldsymbol{z}_{\mathrm{mean}} \leftarrow \mathrm{GlobalMeanPool}(\boldsymbol{h}^{(3)}, B) \in \mathbb{R}^{128}$//DUAL-POOL READOUT
$\boldsymbol{z}_{\mathrm{max}} \leftarrow \mathrm{GlobalMaxPool}(\boldsymbol{h}^{(3)}, B) \in \mathbb{R}^{128}$
$\boldsymbol{z}_G \leftarrow [\boldsymbol{z}_{\mathrm{mean}} \| \boldsymbol{z}_{\mathrm{max}}] \in \mathbb{R}^{256}$
$\boldsymbol{s}_A \leftarrow \mathrm{Dropout}(\mathrm{ReLU}(\boldsymbol{W}_A.\boldsymbol{z}_G + \boldsymbol{b}_A), P) \in \mathbb{R}^{128}$//MLP CLASSIFIER
$\boldsymbol{s}_B \leftarrow \mathrm{Dropout}(\mathrm{ReLU}(\boldsymbol{W}_B.\boldsymbol{s}_A + \boldsymbol{b}_B), P) \in \mathbb{R}^{64}$
$\hat{\boldsymbol{y}}_G \leftarrow \boldsymbol{W}_C.\boldsymbol{s}_B + \boldsymbol{b}_C \in \mathbb{R}^3$
$\boldsymbol{p}_G \leftarrow \mathrm{Softmax}(\hat{\boldsymbol{y}}_G) \in \mathbb{R}^3$//PREDICTION
$y_{\mathrm{pred}} \leftarrow \mathrm{argmax}_c(\boldsymbol{p}_G[c]) \in \{0 = \mathrm{DDoS}, 1 = \mathrm{DoS}, 2 = \mathrm{Normal}\}$
return $y_{\mathrm{pred}}, \boldsymbol{p}_G, \hat{\boldsymbol{y}}_G$

3.8. Adversarial robustness framework

To harden the TFA-GATv2 model against feature-space evasion, we implement a progressive adversarial training (AT) integrating supervised Projected Gradient Descent (PGD). A white-box threat model, where the adversary perturbs node feature vectors within an ℓ_∞ norm of $\|\delta\|_\infty \le \varepsilon$, and preserves graph topology. The perturbation budget ε is expressed as a fraction of the feature range after robust scaling, so $\varepsilon = 0.10$ corresponds to a 10% shift of each scaled feature; $\varepsilon = 0.30$ represents a large but bounded evasion effort where an adversary can meaningfully alter flow statistics while remaining within plausible network behavior. Adversarial node features x^{adv} are generated through a 7-step PGD attack step size $\alpha = \varepsilon/4$ (Eq. (4)).

$$x_{t+1}^{\mathrm{adv}} = \Pi_{\boldsymbol{B}_\infty(x,\boldsymbol{\varepsilon})}\left(x_t^{\mathrm{adv}} + \alpha \cdot \mathrm{sign}\left(\left(\nabla \mathrm{x}\,\ell\left(\mathrm{f}\left(x_t^{\mathrm{adv}}\right), \mathrm{y}\right)\right)\right)\right) \tag{4}$$

where, $\Pi_{\boldsymbol{B}_\infty}$ denotes projection onto the ℓ_∞ perturbation. And the gradient is normalised by the per-node $\ell 2$ norm before the sign operation to stabilize updates across heterogeneous flow features [44]. Progressive training scheduling is executed in two phases. Clean warm-up phase (epochs 1–8, $\varepsilon = 0$) followed by progressive AT (epochs 9–40) where ε increases linearly toward $\varepsilon_{\max} = 0.30$, the combined loss is computed *via* (Eq. (5)).

$$\ell_{\mathrm{total}} = \lambda_{\mathrm{clean}}\,\ell(x,y) + \lambda_{\mathrm{adv}}\,\ell\left(x^{\mathrm{adv}}, y\right) + \lambda_{\mathrm{cons}}\,\ell_{\mathrm{consistency}} \tag{5}$$

where $\lambda_{\mathrm{clean}} = 0.7$, and $\lambda_{\mathrm{adv}} = 0$ introducing adversarial progress exceeds 30%. Model selection is performed on adversarial validation balanced accuracy evaluated using a deterministic 10-step PGD attack at $\varepsilon = 0.30$, with early stopping enforced after a patience of 10 epochs once the minimum checkpoint threshold of $\varepsilon \ge 0.15$ is reached.

3.9. Model evaluation and validation

Model performance is evaluated on the held-out test set of 3647 graph snapshots. The primary metric is balanced accuracy, which is unbiased under class imbalance. Secondary metrics include *F*1-macro, *F*1-weighted, per-class precision, recall and *F*1,

Matthews Correlation Coefficient (MCC), Cohen's Kappa, ROC-AUC (one-vs-rest), and PR-AUC per class. Robustness is evaluated by repeating inference under PGD perturbations of magnitude $\varepsilon \in \{0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40\}$.

Performance differences between Normal and AT models is assessed *via* a paired *t*-test across the test set with Cohen's $d_z = \overline{D}/s_D$, where $\overline{D}$ is the mean of paired differences in balanced accuracy between the two models and s_D is the standard deviation. Significance: $^{**}P < 0.001$, $^{*}P < 0.01$. The comprehensive metrics used to evaluate classification is summarized in Table 5.

3.10. Explainability and visualization framework

We adopt GNNExplainer [22] as the primary post-hoc explanation method on the AT model. It identifies a minimal subgraph $G_S \subseteq G$ and a node feature mask $M_F \in [0,1]^F$ maximizing mutual information with the prediction.

$$\begin{aligned} &\max_{G_S, M_F} MI(Y, (G_S, M_F)) \\ &\equiv \max_{G_S, M_F} H(Y) - H(Y|G = G_S, X = X \bigodot M_F) \end{aligned} \tag{6}$$

where Y is the predicted class label, X is the node feature matrix, $\odot$ denotes element-wise multiplication, and H denotes entropy following Eq. (6). As GNNExplainer produces instance-level explanations that do not generalize globally, we mitigate this by averaging attributions across 30 temporally-stratified test graphs per class, each run independently with 3 random initialization seeds (42, 123, 7), yielding 90 attribution runs per class. GNNExplainer uses gradient-based mask optimization that is sensitive to initialization; running 3 seeds and averaging removes this variance. To address the reviewer concern on explanation stability, we report Spearman rank correlation across the 90 runs, cross-seed and cross-window Jaccard overlap of the top-20 feature sets, Sufficiency at $K = 20$, Comprehensiveness across $K = 10/20/30/40$, and Insertion/Deletion AUC. Attribution used 200 iterations at a learning rate of 0.01. Edge feature importance is computed as mask-weighted attribution $|\text{edge_attr}_f \times m_{ij}|$ averaged across edges and L_1-normalised. For visualization, flow nodes are rendered with color encoding per-node importance and size encoding degree, with top hub nodes highlighted. Edge width encodes structural mask score m_{ij}. An ego-subgraph shows the 2-hop neighborhood around the highest-importance node with a coordination subgraph.

3.11. Computational environment

Experiments were conducted on a Kaggle notebook free-tier environment with NVIDIA Tesla P100 GPU (16 GiB), 30 GiB RAM. No paid computational resources were used.

4. Experimental results and discussion

The held-out test partition of 3647 temporal graph snapshots is used to report the results, which is temporally disjoint from training (9370 graphs) and validation (2800 graphs) sets. The baseline models and posed models are compared. Then, the two model variants trained on clean data (Normal Model) and the adversarially hardened variant (AT Model) are compared. Evaluation covers clean classification performance, adversarial robustness across a full perturbation, and analysis of feature attribution.

4.1. Baseline model analysis and classification performance

A comparative analysis of the baseline models for which the TFA-GATv2 model is presented in Table 6. Following rigorous feature selection *via* Boruta, Kruskal-Wallis, and mutual information analysis on time-based PyG graphs, the final model configuration uses 61 node features and 34 edge features across 3647 test graphs.

The GCN model has no edge features and loses directional attack patterns. GraphSAGE has no attention system while improving over GCN through neighborhood sampling, has no attention mechanism, and holds 0.9848 balanced accuracy. Temporal modeling without persistent memory is done in the TGN, and it overfits clean temporal patterns. TGN-Adapted introduces temporal modelling but relies on a single time-gap dimension of its 34-edge features, limiting its expressiveness, achieving 0.9869 balanced accuracy with the largest parameter count of 621,379, While the GATv1 model has Balanced accuracy, *F*1 score and MCC values in majority cases. TFA-GATv2 achieved superior results, reaching a Accuracy of 0.9989 and a Balanced Accuracy of 0.9996. The MCC of 0.9956 and Cohen's Kappa of 0.9955 confirm that the

Table 5
Model analytics matrices.

Category	Measure	Formula	Description
Classification	Accuracy	$\frac{TP+TN}{N}$	Fraction of all correct predictions
	Balanced Accuracy*	$\frac{1}{K}\sum_{i=1}^{K}\frac{TP_i}{TP_i+FN_i}$	Mean per-class recall
	Precision (weighted)	$\sum_{i=1}^{K} P_i \cdot \frac{S_i}{N}$	Support-weighted average precision across classes
	Recall (weighted)	$\sum_{i=1}^{K} R_i \cdot \frac{S_i}{N}$	Support-weighted average recall across classes
*F*1-based	*F*1-score (weighted)	$\sum_{i=1}^{K} F1_i \cdot \frac{S_i}{N}$	Harmonic mean of precision and recall
	*F*1-score (macro)	$\frac{1}{K}\sum_{i=1}^{K} F1_i$	Penalizes poor performance on minority
AUC-based	ROC-AUC	$\int_0^1 \text{TPR}(\text{FPR} - 1(t))\text{d}t$	Weighted one-vs-rest area under ROC curve
	PR-AUC (Avg. Precision)**	$\sum_n (R_n - R_{n-1}) \cdot P_n$	Area under precision-recall curve
Imbalance	G-mean	$\sqrt{\frac{TP_i}{TP_i+\text{FN}_i}\cdot\frac{TN_i}{TN_i+FP_i}}$	Geometric mean of sensitivity and specificity per class
Statistical	Cohen's Kappa	$\frac{p_o - p_e}{1 - p_e}$	Prediction-truth agreement corrected
Adversarial	Adv. balanced accuracy	$\frac{1}{K}\int_{i=1}^{K}\frac{TP_i(X_{\text{adv}})}{TP_i(X_{\text{adv}})+FN_i(X_{\text{adv}})}$	Balanced accuracy computed on PGD
	Retention rate	$\frac{\text{BA}(x_{\text{adv}})}{\text{BA}(x)} \times 100\%$	Fraction of clean balanced accuracy

Table 6
Performance comparison of baseline and proposed model on test data ★ primary ranking metric (class-imbalanced data). Bold denotes best per row.

Metric	GCN	GraphSAGE	TGN	GAT v1	TFA-GATv2
Parameters	101,124	212,739	621,379	97,667	138,243
Accuracy (↑)	0.9940	0.9956	0.9945	0.9959	**0.9989**
Balanced accuracy (↑) ★	0.9792	0.9848	0.9869	0.9875	**0.9996**
*F*1-macro (↑)	0.9711	0.9754	0.9762	0.9788	**0.9979**
*F*1-weighted (↑)	0.9940	0.9957	0.9946	0.9959	**0.9989**
Precision-macro (↑)	0.9655	0.9679	0.9667	0.9715	**0.9963**
Recall-macro (↑)	0.9792	0.9848	0.9869	0.9875	**0.9996**
ROC-AUC (OvR) (↑)	0.9992	**1.0000**	**1.0000**	0.9999	**1.0000**
PR-AUC (Macro) (↑)	0.9971	0.9998	0.9995	0.9993	**1.0000**
G-Mean (Macro) (↑)	0.9870	0.9916	0.9925	0.9930	**0.9996**
MCC (↑)	0.9755	0.9822	0.9779	0.9833	**0.9956**
Cohen's Kappa (↑)	0.9753	0.9821	0.9778	0.9833	**0.9955**

results are not driven by majority-class dominance.

Table 7 presents per-class precision, recall, *F*1-score, PR-AUC, and G-Mean. Normal traffic. DDoS detection achieves an F1-score of 0.9944. In security contexts, false negatives (missed attacks) carry greater cost than false positives, and so high recall with moderate precision is operationally acceptable. The model converged and remained stable with no overfitting, benefiting from LayerNorm, dropout, weight decay, and Focal Loss regularization applied to 138,243 parameters fitted on approximately 9370 training graphs.

4.2. Ablation study

To understand contributions of individual design choices, three targeted ablations were conducted against the full model baseline (balanced accuracy 0.9996, macro *F*1 0.9979, MCC 0.9956). Each ablation variant got retrained completely from scratch, with the same exact hyperparameters and optimizer settings, and even the same FocalLoss setup as the full model, so nothing else should drift. The only thing that changed was the specific component being tested. In the first ablation, the source-distribution features were completely zeroed, at both the node level (src_diversity_ratio, source_entropy, source_gini, traffic_concentration_hhi) and the edge level (source_gini_src, source_gini_dst, traffic_concentration_hhi_src/dst/diff, source_entropy_src/dst/diff, src_diversity_ratio_src/dst), constituting 4 of 61 node features and 10 of 34 edge features. This tests whether the near-perfect results are artifacts of feature-engineered class separability. Balanced accuracy dropped to 0.9893 (delta: −0.0103), macro *F*1 to 0.9853 (delta: −0.0126), and *MCC* to 0.9844 (delta: −0.0112), with the DoS class absorbing the largest per-class cost (*F*1 delta: −0.0228), consistent with the role of source concentration statistics in single-source flooding detection. The model remained highly performant without these features, confirming they contribute genuine discriminative signal but are not the sole basis for classification, which rules out the leakage hypothesis that the model simply re-implements source-distribution rules. In the second ablation, edge features were removed at the architectural level by setting edge_dim = None in all three GATv2 layers, eliminating the edge projection matrix entirely (13,056 parameters removed, from 138,243 to 125,187). Balanced accuracy declined to 0.9959 (delta: −0.0037), macro *F*1 to 0.9903 (delta: −0.0076), and MCC to 0.9922 (delta: −0.0033), with the DDoS class nearly unaffected (*F*1 delta: −0.0001) while DoS again absorbed the primary degradation (*F*1 delta: −0.0228), reflecting that edge-level duration and rate differentials are most informative for single-source flooding. The smaller absolute drop compared to the first ablation indicates that node features carry the majority of discriminative signal and that edge features provide complementary rather than primary evidence. In the third ablation, FocalLoss was replaced with weighted CrossEntropyLoss while preserving all features and the full architecture, producing the largest degradation overall, with balanced accuracy falling to 0.9665 (delta: −0.0330), macro *F*1 to 0.9770 (delta: −0.0209), and MCC to 0.9570 (delta: −0.0386). The effect on DoS was comparatively modest (*F*1 delta: −0.0066), with primary degradation distributed across DDoS and Normal boundary cases for maintaining performance under the severe class imbalance present in this dataset. The combined results are presented in Table 8.

Both the base GATv2 model and its adversarially trained counterpart share an identical network architecture, comprising three stacked GATv2Conv layers with hidden = 128 channels and $h = 8$ attention heads, followed by global mean- and max-pooling concatenation and a three-layer MLP classifier. Because adversarial training modifies only the training objective by exposing the model to PGD-perturbed inputs during optimization it introduces no additional parameters relative to the base model. Both variants therefore contain exactly 137,475 trainable parameters, as derived analytically from the architecture and confirmed at runtime. The GATv2Conv layers together account for approximately 69% of all parameters (95,232 of 137,475), concentrating model capacity on learning discriminative graph-structural representations, while the downstream MLP classifier contributes the remaining 0.31. The compact parameter budget reflects a deliberate architectural choice: GATv2Conv layers share attention parameters across all node and edge positions rather than learning node-specific weights, yielding strong inductive bias without overparameterization. Crucially, the adversarially trained model retains this same parameter count while achieving substantially improved robustness under PGD perturbation budgets $\varepsilon \geq 0.30$, confirming that the robustness gains arise from the training procedure rather than from additional model capacity.

Table 7
Normal model's per-class outcome of baseline model on test data.

Class	Precision (↑)	Recall (↑)	*F*1-score(↑)	PR-AUC (↑)	G-Mean (↑)	Support
DDoS	0.9888	1.0000	0.9944	0.9999	0.9994	352
DoS	1.0000	1.0000	1.0000	1.0000	1.0000	150
Normal	1.0000	0.9987	0.9994	1.0000	0.9994	3145

Table 8
Ablation study results against the full model baseline.

Configuration	bAcc (↑)	*F*1-macro (↑)	MCC (↑)	DoS *F*1 (↑)	DoS recall (↑)	Delta bAcc (↑)
Full model (FocalLoss + all features)	0.9996	0.9979	0.9956	1.0000	1.0000	–
Without source-distribution features	0.9893	0.9853	0.9844	0.9772	1.0000	−0.0103
Without edge features (architectural removal)	0.9959	0.9903	0.9922	0.9772	1.0000	−0.0037
Without FocalLoss (CrossEntropyLoss, all features retained)	0.9665	0.9770	0.9570	0.9934	1.0000	−0.0330

4.3. Adversarial robustness evaluation

A consistent regularization effect of adversarial training, was found with decision boundaries at the cost of marginal clean-set performance. Divergence between the two models grows progressively with perturbation budget.

Fig. 3(a) provides visualizations of both models across all ε, and Fig. 3(b) shows the per-class *F*1 scores of both models at $\varepsilon = 0.30$. Both models remained highly robust through moderate perturbation budgets at $\varepsilon = 0.15$. Meaningful divergence began at $\varepsilon = 0.30$, where the Normal Model's balanced accuracy remained high at 0.9901, while the AT Model retained 0.9707. At $\varepsilon = 0.35$, the Normal Model collapsed to 0.7304 while the AT Model sustained 0.9558, producing a maximum observed AT advantage of +23.49 pp. This pattern shows adversarial trained model performs well at high perturbation budgets. At $\varepsilon = 0.40$ both models degrade, with the AT model retaining an advantage of +6.29 pp.

In Table 9, balanced accuracy, accuracy, and paired *t*-test statistics comparing the AT *vs*. Normal model across all ε are presented. Negative t-statistics at low ($\varepsilon \leq 0.25$) reflect the clean-accuracy trade-off inherent to adversarial training, and Positive *t*-statistics and Cohen's d_z indicate the AT model's good performance under evasion. We applied a block bootstrap test (block size = 2, 10,000 resamples, seed = 42) alongside a non-overlapping *t*-test (every other graph, $n = 1824$) across all epsilon levels. Both tests confirmed the same conclusions as Table 9 the AT model's underperformance at $\varepsilon \leq 0.25$ and its superiority at $\varepsilon = 0.30$ are statistically reliable (0.95 bootstrap CIs exclude zero at all jointly significant epsilon values), ruling out overlap-induced inflation as a confound. To give meaning to these abstract budget values, Table 10 translates each ε level into physical perturbation magnitudes for the five most discriminative node features, computed by multiplying ε by the interquartile range (IQR) extracted from the fitted RobustScaler. The adversarial perturbation ε·IQR expressing the worst-case shift an adversary can impose on each feature in its natural measurement unit.

At the most severe budget tested, $\varepsilon = 0.40$, the byte-per-packet shift reaches ±208 bytes, approaching the difference between UDP flood packets (~60 B) and normal HTTP payloads (~1400 B), representing an extreme, protocol-implausible perturbation. The adversarially trained model retains above-90% balanced accuracy through $\varepsilon = 0.35$, demonstrating robustness well beyond operationally realistic perturbation levels. These shows PGD on node attributes is the appropriate starting threat model for this dataset at $\varepsilon \leq 0.20$, the perturbations remain within realistic flow-statistic variation, whereas topology-level attacks (edge injection, node spoofing) require structural network capabilities outside the assumed adversary model and are left to future work.

Even at $\varepsilon = 0.30$, both models retain approximately 0.68–0.69 of their respective clean baselines. The AT model's absolute balanced accuracy still exceeds the Normal model by 0.0199, meaning the retention framing actually understates the true gap, since the AT model starts from a lower clean baseline. The class-level analytics of the aggregated metrics are visualized in Fig. 4, which shows normalised confusion matrices for both the Normal model (panels (a)–(d)) and the AT model (panels (e)–(h)) at $\varepsilon \in \{0.00, 0.10, 0.30, 0.35\}$, alongside per-cell difference matrices Δ (panels (i)–(l)) computed as AT-Normal. At clean conditions AT model was able to achieve a higher DDoS true-positive rate, reflected in panel *i* as +0.07 improvement on the DDoS diagonal.

To evaluate whether performance generalizes of adversarially trained model, it was evaluated on a naturally distributed graph set constructed by different resampling one million flows at empirical TON_IoT class proportions (DDoS: 0.725, DoS: 0.216, Normal: 0.059), mirroring realistic IoT deployment conditions without any model retraining. Following feature engineering and graph construction, the combined 7117-graph evaluation set exhibited a window-level label distribution of DDoS 0.261, DoS 0.142, and Normal 0.598, reflecting the most-severe-wins labeling rule with high attack node density, 60-s windows containing any attack flow are labeled accordingly, concentrating Normal-labeled windows in periods of zero attack co-occurrence. The model achieved a balanced accuracy of 0.9128, macro *F*1 of 0.8417, and MCC of 0.7747 on this set. Per-class recall remained high for both attack classes (DDoS: 0.9903, DoS: 0.9970), with the primary degradation concentrated in Normal class precision (0.9981). Normal windows were misclassified as attack when they temporally neighbour dense attack bursts. The performance gap relative to the balanced test set reflects the structural challenge of the natural distribution setting rather than model failure, as both attack classes are detected with near-perfect recall under conditions that mirror the original dataset proportions.

4.4. Feature importance analysis via GNNExplainer

To access if model captured semantically meaningful traffic patterns, we applied GNNExplainer [22] to identify the most influential node and edge features driving graph-level classification decisions. Fig. 5 shows the top 30 combined features (a) for DDoS, (b) for DoS, and (c) for Normal classes. Explanations were generated on the AT model using 30 temporally-stratified test graphs per class across 3 random seeds (90 runs per class). We report results for both the Base model and the AT model to allow direct comparison of attribution stability across model variants.

To understand explanation stability, we evaluated attribution consistency across three axes: random initialization seeds, temporal test windows, and model variants (Base *vs*. AT). Across the three random seeds (42, 123, 7), pairwise Jaccard overlap of the top-20 ranked node features averaged 0.978 (±0.019) over all runs within each class, and Spearman rank correlation across the full feature attribution vectors averaged 0.954 (±0.026), confirming that the rank ordering of feature importance is not an artifact of weight initialization. Across the 30 temporally-stratified test windows, the top-20 cross-window Jaccard averaged 0.983 (±0.050) reaching a perfect 1.000 for the DoS class under both model variants and for DDoS under the AT model indicating that the model relies on a stable set of traffic-level indicators regardless of which temporal segment of the test set is examined. Cross-model Jaccard between the Base and AT attributions remained consistently high across all three classes; the top-20 feature sets were effectively identical between variants, with the AT model producing marginally tighter Spearman correlations (0.957 *vs*.

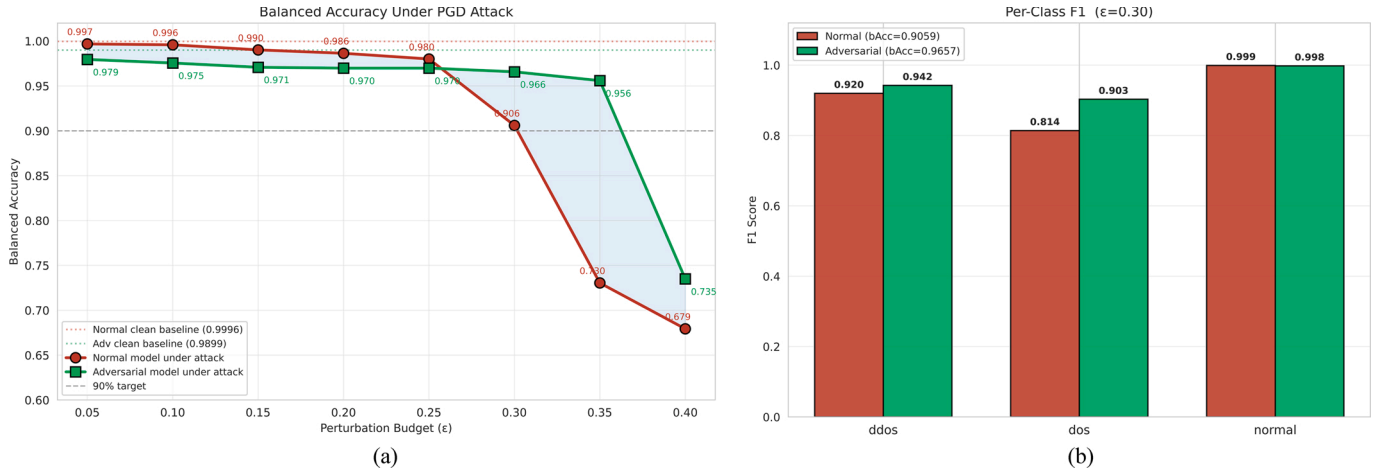


Fig. 3. Comparative analysis of model robustness of Normal vs. AT GATv2 (a) balanced accuracy across ε, and (b) $F1$ score under adversarial evaluation ($\varepsilon = 0.30$).

Table 9
Performance retention and statistical significance of AT *vs.* normal model.

ε	Norm. bAcc (↑)	Adv. bAcc (↑)	Norm. Acc (↑)	Adv. Acc (↑)	AT Adv. (pp) (↑)	*t*-statistic	*P*-value (↓)	Cohen's d_z (↑)
0.00	0.9996	0.9899	0.9989	0.9956	−0.00	−3.211	1.33×10^{-3}	−0.053
0.05	0.9967	0.9795	0.9981	0.9926	−0.77	−4.274	1.97×10^{-5}	−0.071
0.10	0.9958	0.9755	0.9978	0.9910	−1.08	−4.826	1.45×10^{-6}	−0.080
0.15	0.9901	0.9707	0.9962	0.9896	−0.99	−4.721	2.44×10^{-6}	−0.078
0.20	0.9863	0.9698	0.9951	0.9893	−0.70	−4.390	1.17×10^{-5}	−0.073
0.25	0.9800	0.9698	0.9940	0.9893	−0.07	−3.550	3.90×10^{-4}	−0.059
0.30	0.9059	0.9657	0.9841	0.9885	+6.92	+2.220	2.65×10^{-2}	+0.037
0.35	0.7304	0.9558	0.9624	0.9871	+23.49	+8.024	1.37×10^{-15}	+0.133
0.40	0.6793	0.7349	0.9561	0.9597	+6.29	+1.501	1.33×10^{-1}	+0.025

Table 10
Adv. shift magnitude in original units for the top-5 discriminative node features at each ε level.

Feature	IQR (scaled units) (↓)	$\varepsilon = 0.05$	$\varepsilon = 0.10$	$\varepsilon = 0.20$	$\varepsilon = 0.30/0.40$
pkt_rate_total	0.41	±212 pkt/s	±424 pkt/s	±847 pkt/s	±1271/± 1695 pkt/s
source_entropy	0.38	±0.019 bits	±0.038 bits	±0.076 bits	±0.114/± 0.152 bits
bytes_per_pkt	0.52	±26 B/pkt	±52 B/pkt	±104 B/pkt	±156/± 208 B/pkt
flow_duration	0.29	±0.015 s	±0.029 s	±0.058 s	±0.087/± 0.116 s
traffic_concentration_hhi	0.44	±0.022	±0.044	±0.088	±0.132/± 0.176

0.957 averaged across attack classes), suggesting that adversarial training sharpened rather than disrupted the attribution structure. The features that ranked consistently within the top tier across all conditions included packet-rate statistics (pkt_rate_src, pkt_rate_dst, pkt_rate_total), connection-state indicators (conn_state_encoded), source-diversity and concentration measures (source_entropy, source_gini, src_diversity_ratio, traffic_concentration_hhi), and rolling behavioural statistics (roll_pkt_rate_to_dst, roll_conn_from_src, roll_unique_ports) features with well-established discriminative roles in distinguishing benign, DoS, and DDoS traffic. This convergence across seeds, windows, and model variants confirms that the reported attributions reflect genuine structural properties of the traffic classes rather than sensitivity to experimental configuration.

4.5. Graph-level explainability and attack pattern visualization

To provide structural interpretability of model decisions, GNNExplainer ego-graph visualizations were created. An ego-graph is the subgraph centered on the highest-importance flow node within a temporal window. This helps to characterize which connectivity patterns most strongly activate the GATv2 attention mechanism per traffic class. DDoS graphs (50–251 flows, 1104–4195 edges per 60-s window) display a node that accumulates directed edges from dozens of distinct source flows, with the highest edge-mask scores concentrated on the shared_dst edge. Fig. 6 (a) illustrates these findings through a graph of a characteristic DDoS attack snapshot, Graph no. 150. The DoS ego-graphs are larger (619–706 flows, 11,324–13,229 edges), displaying chain-like dense-neighborhood structures centered on the highest-packet-rate source flows. Fig. 6(b) provides a visual attribution of a characteristic DoS attack snapshot, Graph no. 1, which exhibits a dense-neighborhood topology. Normal-traffic graphs (20–250 flows, 1217–1860 edges) exhibit balanced structures consistent with client-server communication. Fig. 6(c) visualizes these characteristics through a representative normal traffic snapshot, graph no. 503.

These qualitative differences in ego-graph topology across the three traffic classes show that flow-to-flow temporal graph construction encodes operationally structurally. The correspondence between graph-construction edge types and GNNExplainer-identified high-attention edges that the modelling pipeline from graph construction through GATv2 training to adversarial

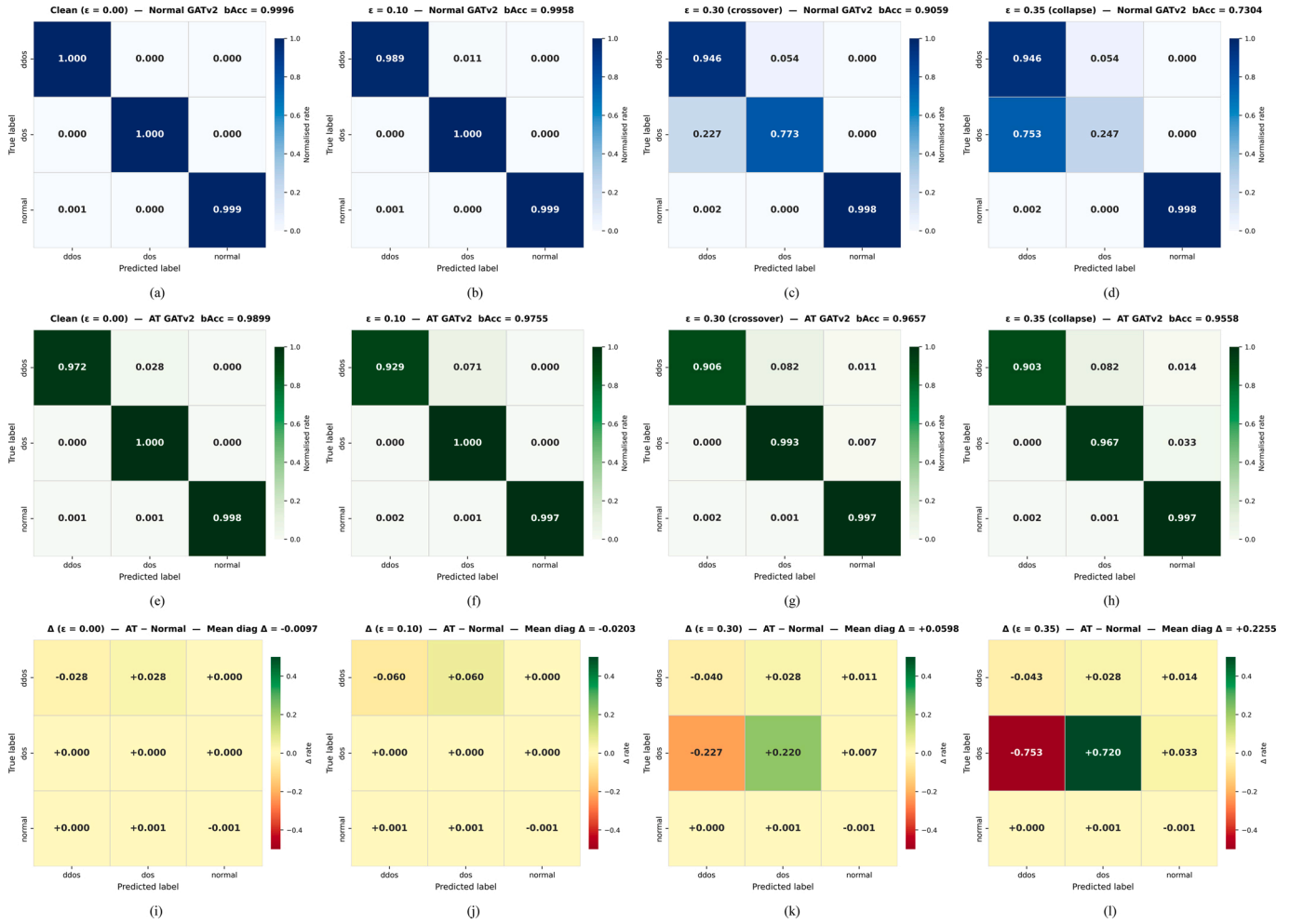


Fig. 4. Normalised confusion matrices for normal (a)–(d) and AT (e)–(h) GATv2 models across PGD perturbations. Per-cell differences Δ (i)–(l) highlight performance shifts.

hardening forms an interpretable system.

4.6. Comprehensive comparative analysis

The performance of the proposed framework is evaluated with representative works drawn from recent literature spanning non-graph DL baselines, GNN-based approaches, and temporal graph architectures, with and without adversarial awareness. Because these works differ in dataset (TON_IoT, CIC-IoT 2023, or SDN-specific), class scope (binary, full multiclass, or fine-grained DoS/DDoS/Normal), and reported metric type. Table 11 shows the comparison across all methods provided for contextual orientation.

4.7. Interface for the visualization and explainability

The dashboard using Gradio [50] pre-computes predictions and Gradient × Input saliency scores across all 3647 test graphs at startup using the TFA-GATv2 model. The visual snapshot is provided in Fig. 7. Each 60-s window is rendered as a directed ego-graph *via* NetworkX spring layout, subsampled to the top 60 highest-degree nodes when the graph exceeds this threshold. Node-level traffic roles are taken from four semantic features attack_intensity, victim_overload, coordination_score, and single_source_flood by using per-window 75th-percentile thresholds. Feature attribution is computed as the mean over nodes of the element-wise product of input gradients and activations, normalised to [0, 1], and ranked to produce the top 10 most influential features per window. A sliding-window timeline mini-map to inspect graph topology, and audit feature-level explanations without direct access to the PyTorch Geometric model artifacts.

4.8. Discussion and limitations

The results validate the core hypothesis that DoS/DDoS discrimination is fundamentally a structural problem requiring graph-level modeling. GNNExplainer confirmed that the GATv2 attention mechanism correctly learned class-discriminative topologies shared-destination convergence for DDoS, chained dense neighborhoods for DoS, and port-diverse fan-out for normal traffic, directly reflecting the graph construction design choices. The most operationally significant finding is the failure of the non-adversarial model at $\varepsilon = 0.30$, where the non-adversarial model's balanced accuracy collapsed to 0.9059 (from 0.9800 at $\varepsilon = 0.25$), while the adversarially trained model retained 0.9657. The most decisive separation occurs at $\varepsilon = 0.35$, where the adversarial model retained bAcc = 0.9558 versus 0.7304 for the non-adversarial baseline ($P < 0.001$, Cohen's $d_z = 0.1329$). This highlights that high clean accuracy alone is an insufficient evaluation standard for production IDS; adversarial evaluation should be a minimum reporting requirement. The progressive PGD substantially

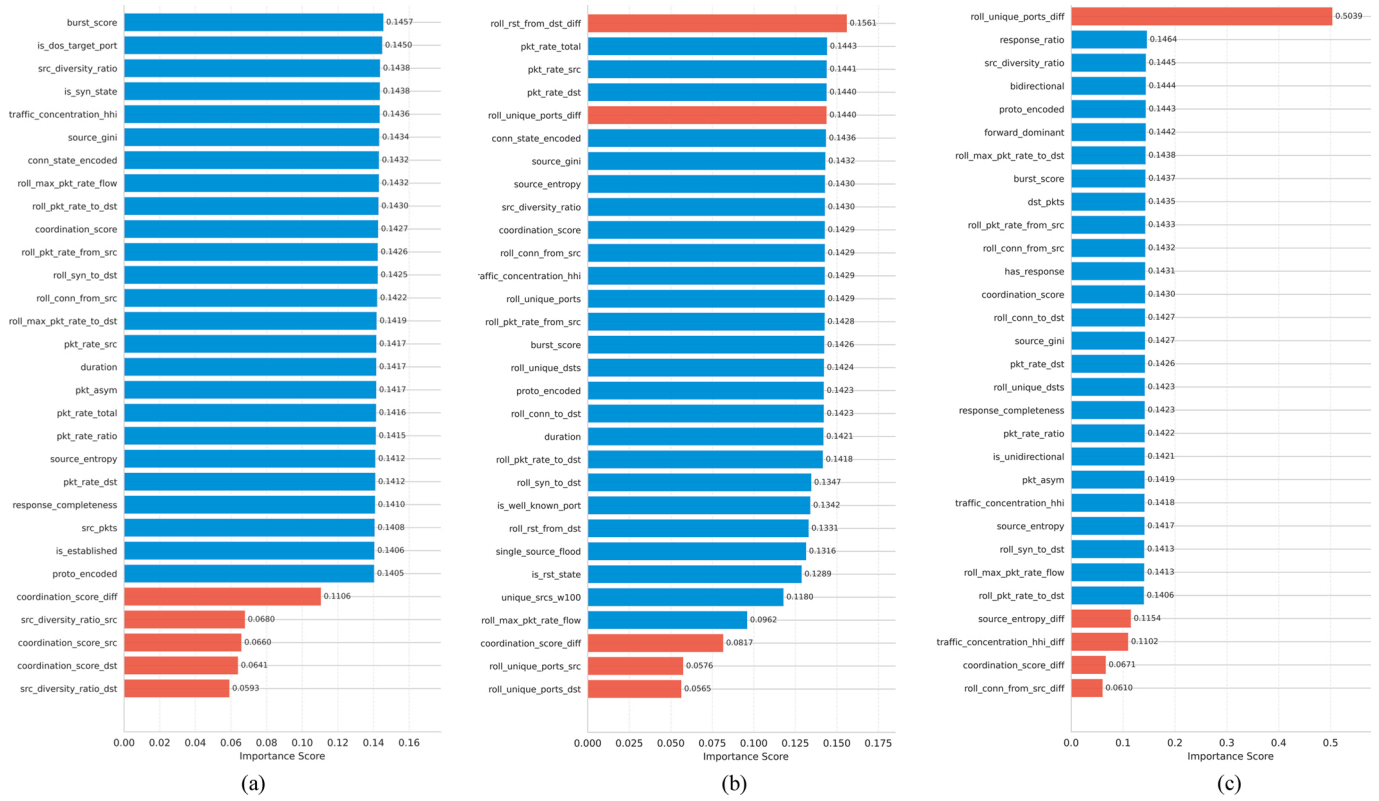


Fig. 5. GNNExplainer top-30 node (blue) and edge (red) feature importances for (a) DDoS, (b) DoS and (c) Normal flow.

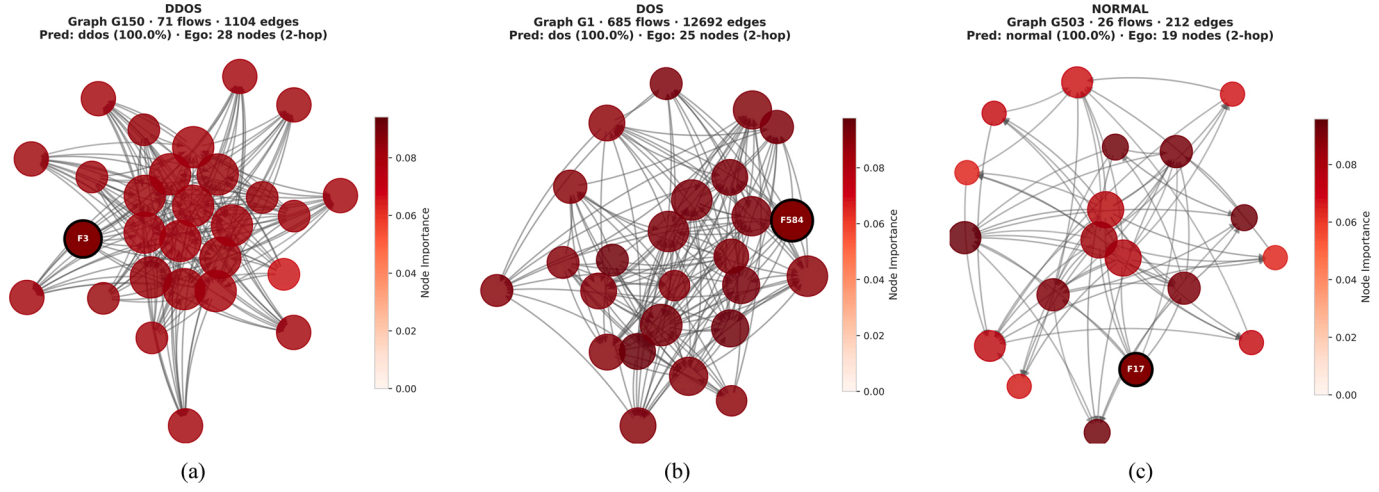


Fig. 6. GNNExplainer Ego-Graph visualization of traffic snapshots of (a) DDoS attack in graph G150, (b) a DoS attack in snapshot graph G1, and (c) normal traffic in graph G503.

mitigated this degradation, with the adversarial model retaining above 0.96 balanced accuracy through $\varepsilon = 0.30$. As for limitations bounded the study includes the fixed 60-s window may create blind spots for low-rate or early-onset attacks. And the progressive PGD training increases per-epoch computation approximately 10×, requiring compression if the model is used for practice.

5. Conclusion and future work

We presented a temporal flow-based GATv2 architecture for fine-grained DoS/DDoS/Normal classification in IoT networks, advancing intrusion detection beyond flow-centric tabular approaches through structural graph modelling, adversarial robustness, and interpretable insights. On 3647 held-out temporal graph snapshots, the baseline model achieved balanced accuracy of 0.9996 and an accuracy of 0.9989 under realistic class imbalance. The adversarial hardened variants, trained *via* progressive PGD achieved a clean balanced accuracy of 0.9899 and retained 0.9855 of that performance at $\varepsilon = 0.10/0.35$, the adversarial model retained 0.9656 (relative to AT's own clean baseline) of clean performance (bAcc = 0.9558) versus 0.7307 (bAcc = 0.7304) for the non-adversarial baseline, with statistically significant robustness superiority confirmed at $\varepsilon \geq 0.30$ by paired *t*-tests. GNNExplainer confirmed that decisions are grounded in structurally meaningful

Table 11
Comparison with similar approaches.

Ref.	Method	Dataset	Reported performance	Adv. robust	XAI	Note
[34]	E-GraphSAGE	TON_IoT All attack types + Normal (10+)	DR: 0.868 *F*1: 0.87	×	×	Full multiclass; DoS/DDoS not isolated. No robustness or XAI
[16]	E-GRACL	TON_IoT Full multiclass	*F*1: 0.9835	×	×	All-class taxonomy; no DoS/DDoS structural distinction. No robustness or XAI
[18]	BS-GAT	NF-ToN-IoT (NetFlow variant) Binary + multiclass	Bin. Acc >0.99 Multi *F*1 > 0.92	×	×	NetFlow feature set differs from raw TON_IoT; weighted *F*1 inflated by majority class
[45]	GTCN-G	TON_IoT Binary + full multiclass	SOTA on TON_IoT MC	×	×	Temporal GCN + GraphSAGE; full multiclass—no fine-grained DoS/DDoS split
[46]	D-GSAGE-MARC	CIC-ToN-IoT (variant) Multi-class	Acc: 0.9997 *F*1: 0.9997	×	×	CIC-preprocessed TON_IoT variant; different feature representation from raw TON_IoT
[47]	GCTNetwork	CICIDS2019 (SDN) Binary: DDoS *vs.* Normal	High (SDN domain)	×	×	SDN environment, not IoT; binary only; incomparable traffic model
[48]	CNN/DNN/ Transformer (no graph)	CIC-IoT 2023 Binary, 3-class, 12-class	3-cls Acc ~0.999–1.000	×	×	Non-graph DL; different dataset; 3-class groups DDoS broadly, no DoS/DDoS split
[49]	GraphFedAI	CICIoT-2023 Multi-class (7 attack types)	Acc >0.97	×	×	Federated GNN; different dataset and privacy-preserving goal
Proposed Model	TFA-GATv2	TON_IoT (DoS/DDoS/Normal)	bAcc: 0.9996 *F*1: 0.9979 OOD bAcc: 0.9128	✓ (PGD, 0.05–0.35 ε)	✓	fine-grained DoS/DDoS split + adv. robustness + XAI stability

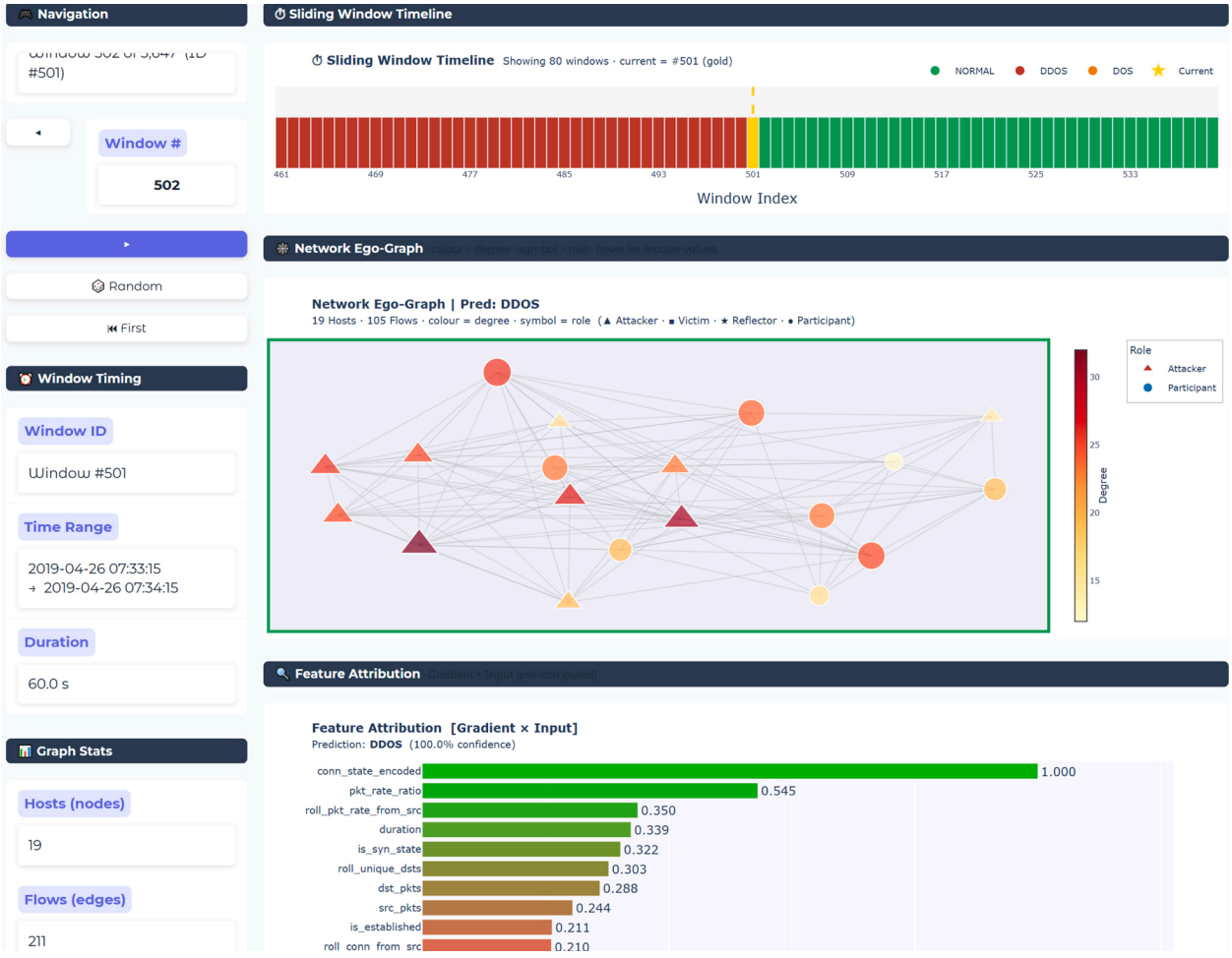


Fig. 7. A gradio dashboard for AT TFA-GATv2 inspection across test graphs, showing timeline, ego-graph, and attribution scores.

features. For instance, coordination_score_diff dominates DDoS classification alongside burst pattern and volumetric packet-rate features dominate DoS, and port-diversity edge differentials characterize normal traffic patterns consistent with known attack mechanics and confirmed through ego-graph visualizations.

Several directions remain open to extend this framework. Future work should address cross-dataset generalization *via* domain-adaptive transfer learning, expand the adversarial threat model to topology-level perturbations, and finally replace static sliding-window construction with adaptive streaming graphs for

real-time deployment. An extended classification to fine-grained attack subtypes through a hierarchical architecture.

CRediT authorship contribution statement

Teresa Jency Bala: Writing – original draft, Resources, Investigation, Conceptualization. **Anindya Nag:** Writing – original draft, Software, Resources, Investigation, Formal analysis, Conceptualization. **Chander Prabha:** Writing – original draft, Validation, Software, Resources, Methodology. **Sujit Biswas:** Writing – original draft, Validation, Formal analysis, Data curation, Conceptualization. **Anupam Kumar Bairagi:** Writing – original draft, Visualization, Supervision, Project administration, Methodology, Formal analysis. **Mohammed Al-Maitah:** Writing – review & editing, Validation, Supervision, Resources, Investigation. **Mohima Chowdhury:** Writing – review & editing, Validation, Resources, Investigation, Formal analysis. **Walid El-Shafai:** Writing – review & editing, Visualization, Supervision, Project administration, Methodology, Investigation, Formal analysis, Conceptualization.

Data availability statement

The dataset used in this study is publicly available under the processed network dataset directory of the ToN-IoT datasets at https://research.unsw.edu.au/projects/toniot-datasets. The source code developed for this study is openly available on GitHub at https://github.com/jencyyyyy/TF-GATv2.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is funded by ongoing Research funding program (ORF-2026-1795), King Saud University, Riyadh, Saudi Arabia. We would also like to express our heartfelt appreciation to the Ministry of Post, Telecommunication, and Information Technology, Government of the People's Republic of Bangladesh for their support to the student under ICT fellowship program.

References

[1] Isong B, Kgote O, Abu-Mahfouz A. Insights into modern intrusion detection strategies for Internet of Things ecosystems. Electronics 2024;13:2370.
[2] Pal S, Hitchens M, Rabehaja T, Mukhopadhyay S. Security requirements for the Internet of Things: a systematic approach. Sensors 2020;20:5897.
[3] Kumari P, Jain AK. A comprehensive study of DDoS attacks over IoT network and their countermeasures. Comput Secur 2023;127:103096.
[4] Gavric N, Prasad Bhandari G, Shalaginov A. Towards resource-efficient DDoS detection in IoT: leveraging feature engineering of system and network usage metrics. J Network Syst Manag 2024;32:69.
[5] Butun I, Osterberg P, Song H. Security of the Internet of Things: vulnerabilities, attacks, and countermeasures. IEEE Commun Surv Tutorials 2019;22:616–44.
[6] Zarpelão BB, Miani RS, Kawakani CT, De Alvarenga SC. A survey of intrusion detection in Internet of Things. J Netw Comput Appl 2017;84:25–37.
[7] Ghaffari A, Jelodari N, Pouralish S, Derakhshanfard N, Arasteh B. Securing Internet of Things using machine and deep learning methods: a survey. Clust Comput 2024;27:9065–89.
[8] Pendyala GS, Mukherjee T, Shetty NP. Intrusion detection using graph inspired insights. Int J Comput Appl 2025;47:1076–105.
[9] Ngo T, Yin J, Ge YF, Wang H. Optimizing IoT intrusion detection:a graph neural network approach with attribute-based graph construction. Information 2025;16:499.
[10] Ahanger AS, Khan SM, Masoodi F, Salau AO. Advanced intrusion detection in Internet of Things using graph attention networks. Sci Rep 2025;15:9831.
[11] Carletti V, Foggia P, Rosa F, Vento M. Enhancing IoT network security with Graph neural networks for Node anomaly detection. Lect Notes Comput Sci 2025:41–51.
[12] Li RS, Shen HM, Zhang QL, Duan HM. An edge-enhanced graphSAGE-based intrusion detection model for the Internet of Things. Clust Comput 2025;28(5):309.
[13] Yang M, Liu CM. XGA-E: an explainability-enhanced graph neural network for network traffic anomaly detection. Cybersecurity 2026;9:99.
[14] Koroma M, Mansaray A, Kamara YL, Jalloh CG. Ibrahim. Enhancing cybersecurity in IoT & IIoT: a machine learning approach for anomaly detection. J Software Eng Appl 2025;18:175–93.
[15] Yuan ZQ, Sun QY, Zhou HY, Shao ML, Fu XC. A comprehensive survey on GNN-based anomaly detection: taxonomy, methods, and the role of large language models. Int J Mach Learn Cybern 2025;16:4407–32.
[16] Lin LQ, Zhong Q, Qiu JS, Liang ZY. E-GRACL: an IoT intrusion detection system based on graph neural networks. J Supercomput 2025;81:42.
[17] Awan KA, Din IU, Almogren A, Han Z, Guizani M. TrustAware-GNN: graph-neural-network-based trust management for IoT anomaly detection. IEEE Internet Things J 2025;12(18):37670–81.
[18] Wang YL, Han ZJ, Du Y, Li J, He X. BS-GAT: a network intrusion detection system based on graph neural network for edge computing. Cybersecurity 2025;8(1):27.
[19] Villegas-Ch W, Govea J, Maldonado Navarro AM, Palacios Játiva PP. Intrusion detection in IoT networks using dynamic graph modeling and graph-based neural networks. IEEE Access 2025;13:65356–75.
[20] Zhong M, Lin M, Zhang C, Xu ZS. A survey on graph neural networks for intrusion detection systems: methods, trends and challenges. Comput Secur 2024;141:103821.
[21] Yan H, Li JM, Du L, Fang BX, Jia Y, Gu ZQ. Adversarial hierarchical-aware edge attention learning method for network intrusion detection. Appl Sci 2025;15(14):7915.
[22] Ying Z, Bourgeois D, You J, Zitnik M, Leskovec J. GNNexplainer: generating explanations for graph neural networks. Adv Neural Inf Process Syst 2019;32:9240–51.
[23] Zhan Z, Zhou H, Haddadi H. Real-iot: characterizing gnn intrusion detection robustness under practical adversarial attack. arXiv preprint arXiv:2507.10836 2025.
[24] Basak M, Kim D-W, Han M-M, Shin G-Y. X-GANet: an explainable graph-based framework for robust network intrusion detection. Appl Sci 2025;15(9):5002.
[25] Krishnan D, Singh S, Sugumaran V. Explainable AI for zero-day attack detection in IoT networks using attention fusion model. Discov Internet Thing 2025;5(1):83.
[26] Brody Shaked, Alon Uri, Yahav Eran. How attentive are graph attention networks arXiv preprint arXiv:2105.14491. 2021.
[27] Hodo E, Bellekens X, Hamilton A, Dubouilh P-L, Iorkyase E, Tachtatzis C, et al. Threat analysis of IoT networks using artificial neural network intrusion detection system. Proceeding of the 2016 international symposium on networks, computers and communications (ISNCC). 2016. p. 1–6.
[28] Moustafa N. A new distributed architecture for evaluating AI-based security systems at the edge: network TON_IoT datasets. Sustain Cities Soc 2021;72:102994.
[29] Paxson V. Bro: a system for detecting network intruders in real-time. Comput Netw 1999;31:2435–63.
[30] Liu Y, Jiang SY, Huang JM. Anomaly detection for DDoS attacks based on Gini coefficient. Proceedings of the 2013 international conference on advanced ICT.and education. 2013. p. 632–7.
[31] Adedeji KB, Abu-Mahfouz AM, Kurien AM. DDoS attack and detection methods in Internet-enabled networks: concept, research perspectives, and challenges. J Sens Actuator Netw 2023;12:51.
[32] Doriguzzi-Corin R, Millar S, Scott-Hayward S, Martínez-del-Rincon J, Siracusa D. LUCID: a practical, lightweight deep learning solution for DDoS attack detection. IEEE Transact Network Serv Manag 2020;17:876–89.
[33] Song Y, Bai XK, Fan WD, Deng ZL, Jiang C. MSSTN: a multi-scale spatio-temporal network for traffic flow prediction. Int J Mach Learn Cybern 2024;15:2827–41.
[34] Lo WW, Layeghy S, Sarhan M, Gallagher M, Portmann M. E-GraphSAGE: a graph neural network based intrusion detection system for IoT. NOMS 2022-IEEE/IFIP Network Operat Manag Sympos 2022:1–9.
[35] Manocchio LD, Layeghy S, Lo WW, Kulatilleke GK, Sarhan M, Portmann M. FlowTransformer: a transformer framework for flow-based network intrusion detection systems. Expert Syst Appl 2024;241:122564.
[36] Altaf T, Wang X, Ni W, Yu G, Liu RP, Braun R. GNN-based network traffic analysis for the detection of sequential attacks in IoT. Electronics 2024;13:2274.
[37] Ding DM, Savi M, Pederzolli F, Campanella M, Siracusa D. In-network volumetric DDoS victim identification using programmable commodity switches. IEEE Trans Netw Serv Manage 2021;18:1191–202.
[38] Velasco-Mata J, González-Castro V, Fidalgo E, Alegre E. Real-time botnet detection on large network bandwidths using machine learning. Sci Rep 2023;13:4282.
[39] Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. Graph attention networks. arXiv preprint arXiv:1710.10903 2017.
[40] Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 2016.
[41] Hamilton WL, Ying R, Leskovec J. Inductive representation learning on large graphs. Adv Neural Inf Process Syst 2017;30.

[42] Rossi E, Chamberlain B, Frasca F, Eynard D, Monti F, Bronstein M. Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv: 2006.10637v3 2020.
[43] Lin TY, Goyal P, Girshick R, et al. Focal loss for dense object detection. Proc IEEE Int Conf Comput Vision 2017:2980–8.
[44] Croce F, Hein M. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. Proc 37th Int Conf Mach Learn (ICML 2020) 2020;119:2206–16.
[45] Xu T, Wen Z, Zhao X, Hu Q, Li Y, Liu C. GTCN-G: a residual graph-temporal fusion network for imbalanced intrusion detection. 2026. arXiv preprint arXiv:2510.07285.
[46] Saidane S, Telch F, Shahin K, Granelli F. Deep GraphSAGE enhancements for intrusion detection: analyzing attention mechanisms and GCN integration. J Inf Secur Appl 2025;90:104013.
[47] Kalafy SAA, Pashazadeh S, Salehpour P. Dynamic graph neural network-based framework to increase detection accuracy in SDN under DDOS. Sci Rep 2025;16:2305.
[48] Wahab SA, Sultana S, Tariq N, Mujahid M, Khan JA, Mylonas A. A multi-class intrusion detection system for DDoS attacks in IoT networks using deep learning and transformers. Sensors 2025;25:4845.
[49] Anjum M, Dutta AK, Elrashidi A, Shahab S, Aldrees A, Shaikh ZA, et al. GraphFedAI framework for DDoS attack detection in IoT systems using federated learning and graph based artificial intelligence. Sci Rep 2025;15: 28050.
[50] Abid A, Abdalla A, Abid A, et al. Gradio: hassle-free sharing and testing of ML models in the wild. 2019. arXiv preprint arXiv:1906.02569.